



Лектор: Мещерин Илья Семирович 94го года рождения

Для вас техали: *Потапов Станислав,
Сысоева Александра,
Бадоян Пётр,
Баронов Михаил,
Белецкий Иван,
Артем Шарипов,
Обухов Леонид,
Еникеев Арнольд,
Оля Терехова*

Содержание

1	Now, we're going to start C++ introduction	3
2	Основные типы и операции над ними	5
3	Объявления, определения и области видимости	8
4	Выражения (expressions) и операторы	10
5	Управляющие инструкции (control statements)	14
6	Понятия CE, RE и UB	18
7	Указатели	21
8	Виды памяти	25
9	Массивы	28
10	Функции	30
11	Ссылки	33
12	Константы	36
13	Приведения типов	40
14	Классы и структуры, модификаторы доступа	42
15	Конструкторы и деструкторы	46
16	Правило трёх	50
17	Константные и статические методы	52
18	Наследование	55
19	Приведения типов при наследовании(в случае публичного наследования)	58
20	Шаблоны	60
21	Шаблоны с числовыми параметрами	61
22	Полиморфизм и виртуальные функции	62
23	Абстрактные классы и чисто виртуальные функции	63
24	Исключения	65
25	Идиома RAII	68
26	Контейнеры	73
27	Итераторы, категории итераторов	75

28 Move-семантика	78
29 Ключевое слово auto	82
30 Лямбда-функции	86
31 Constexpr-функции и переменные	91

1 Now, we're going to start C++ introduction

in this concept, we are going to discuss about what is C++, okay? [source of meme](#)

Понятие компилятора.

Разница между компилируемыми и интерпретируемыми языками.

Примеры компиляторов C++. Запуск компилятора из командной строки.

Пример простейшей программы и ее компиляция, файл a.out.

Простейший ввод-вывод, использование cin и cout.

Компилятор — это транслятор, который осуществляет перевод исходной программы в эквивалентную ей объектную программу на языке машинных команд или языке ассемблера.

Компилируемые языки требуют предварительного преобразования кода, написанного на языке программирования, в последовательность машинных команд. Этот процесс называется компиляцией. Получаемые на выходе бинарные файлы можно запускать непосредственно на машине под управлением операционной системы.

Интерпретируемые языки не требуют компиляции, а исходный код транслируется в машинные команды непосредственно во время работы приложения при помощи вспомогательной программы - интерпретатора. Производительность приложений, написанных на интерпретируемых языках, ниже, чем на компилируемых.

Примерами компиляторов для C++ являются GCC, clang, Visual C++. Рассмотрим запуск компилятора из Unix-терминала на примере GCC.

```
1 g++ main.cpp -o a.out
```

Дополнительные флаги, которые можно использовать при компиляции.

- -std — флаг для определения используемой версии стандарта C++. Пример: -std=c++20
- -O — флаг для выбора оптимизации при компиляции. Пример: -O2, -O0
- -fsanitize — флаг для выбора санитайзера. Пример -fsanitize=address
- -W — флаг для выбора варнингов. Пример -Wall, -Wunused
- -g — флаг для добавления в бинарный файл дебаг-символов, необходимых для отладки программ с помощью дебаггеров, например gdb или lldb

Для стандартного ввода и вывода из потока существует стандартная библиотека iostream. Пример простейшей программы, использующей ввод и вывод:

```
1 #include <iostream>
2
3 int main() {
4     char s;
5
6     std::cin >> s; //input from input stream into a variable s
7
8     s = '&';
9
10    std::cout >> s; // output to the standard output stream
11 }
```

Можно задать потоки ввода и вывода при запуске программы из терминала.

```
1    g++ main.cpp -o a.out
2    touch out.txt
3    touch input.txt
```

Теперь чтобы при запуске программы a.out на вход подавалось содержимое input.txt, а выход в out.txt, нужно всего лишь...

```
1    cat input.txt | ./a.out > out.txt
```

2 Основные типы и операции над ними

Понятия статической и динамической типизации. Целочисленные типы. Беззнаковые версии этих типов. Тип `size_t`. Типы с фиксированной шириной и их беззнаковые версии. Логический и символьный типы. Логические операции. Типы с плавающей точкой. Понятия `integer/floating point promotion` и `standard conversion`. Понятие литерала, литеральные суффиксы. Префиксы `0` и `0x` для чисел. Типы `std::string`, `std::vector`, `std::set`, `std::map`, основные операции над ними.

Статическая типизация — типы известны на этапе компиляции, что позволяет заранее произвести проверку типов. Статическая типизация свойственна компилируемым языкам.

Динамическая типизация — типы определяются уже в процессе выполнения программы, что с одной стороны упрощает написание кода, позволяя не заботиться о типах, с другой стороны может привести к ошибкам, так как усложняет предварительную проверку типов. Динамическая типизация свойственна интерпретируемым языкам.

Целочисленные типы.

Type specifier	Equivalent type	Width in bits by data model				
		C++ standard	LP32	ILP32	LLP64	LP64
<code>signed char</code>	<code>signed char</code>	at least 8	8	8	8	8
<code>unsigned char</code>	<code>unsigned char</code>					
<code>short</code>	<code>short int</code>	at least 16	16	16	16	16
<code>short int</code>						
<code>signed short</code>						
<code>signed short int</code>						
<code>unsigned short</code>	<code>unsigned short int</code>					
<code>unsigned short int</code>						
<code>int</code>	<code>int</code>	at least 16	16	32	32	32
<code>signed</code>						
<code>signed int</code>						
<code>unsigned</code>	<code>unsigned int</code>					
<code>unsigned int</code>						
<code>long</code>	<code>long int</code>	at least 32	32	32	32	64
<code>long int</code>						
<code>signed long</code>						
<code>signed long int</code>						
<code>unsigned long</code>	<code>unsigned long int</code>					
<code>unsigned long int</code>						
<code>long long</code>	<code>long long int</code> (C++11)	at least 64	64	64	64	64
<code>long long int</code>						
<code>signed long long</code>						
<code>signed long long int</code>						
<code>unsigned long long</code>	<code>unsigned long long int</code> (C++11)					
<code>unsigned long long int</code>						

Целочисленные типы данных позволяют хранить целые числа. Максимальные и минимальные числа, которые позволяет хранить тот или иной тип, зависят от размера, занимаемого этим типом в памяти. А также от того, является этот тип знаковым или беззнаковым. Переполнение знакового

типа (вверх или вниз) является UB. Переполнение беззнакового типа UB не является: операции работают как по модулю числа возможных хранимых значений. Например, для беззнакового 32-битного типа $0 - 1 = 2^{32} - 1$

Существуют также типы с фиксированной шириной. Их размер (в битах) задается напрямую. Их примеры:

- `int8_t` / `uint8_t`
- `int16_t` / `uint16_t`
- `int64_t` / `uint64_t`

Логический и символьный типы. Типы `char` и `bool` оба занимают 1 байт (`bool` из-за особенностей выравнивания памяти). Однако в `bool` хранится значение `false` или `true`. А в `char`: от 0 до 255, если он беззнаковый.

Логические операции:

```
1 int main() {
2     bool a = true;
3     bool b = false;
4
5     !a; // -> not(a)
6     a && b; // -> a and b
7     a || b; // -> a or b
8 }
```

Порядок парсинга логических операций:

`not` -> `and` -> `or`

Типы с плавающей точкой. Данные типы хранятся немного иначе, чем рассмотренные ранее. В них один бит выделен на знак, часть бит на мантиссу, часть бит на экспоненту. То есть хранится фиксированное количество значащих цифр, вследствие чего абсолютная точность хранения числа уменьшается, при увеличении модуля этого числа.

Основные типы это: `float` - 32 bit, `double` - 64 bit. Так же есть тип `long double`, занимающий 128 бит

Integer/floating point promotion. Standard conversion Существует неявное приведение между всеми основными типами. Одни из них работают нормально, другие же не особо. Нормально приведение работает, если диапазон одного типа полностью входит в диапазон типа, в который мы хотим привести. В обратном случае есть некоторые проблемки, нужно читать стандарт.

Например, `int` без проблем неявно конвертируется в `long long`, а `float` в `double`. При включённых предупреждениях, компилятор предупреждает о неявных преобразованиях, теряющих точность.

Литералы. Литералы это выражения, которые интерпретируются как константы некоторого типа. Например:

```
1 1, 'a', 2.5f, true, nullptr, "abacaba"
```

По умолчанию, литералы из цифр без точки - `int`, с точкой - `double`. Если мы хотим, чтобы компилятор считал данный литерал другим типом, нужно использовать суффикс.

```
1 111, 25u, 2.5f, 0.00041d
```

"l" означает `long long`, "u" означает `unsigned`, "f" — `float`, "ld" — `long double`

Префиксы 0 и 0x Префикс 0 означает запись числа в 8-ричной системе счисления. 0x - в 16-ричной. Например:

```
1 42 = 052 = 0x2a
```

std::string Данный тип позволяет хранить строки из char-ов. [Полное описание данного контейнера на cppreference](#)

Позволяет обращаться к строке по индексу, конкатенировать две строки, удалять и добавлять символ в произвольное место.

std::vector Шаблонный класс, позволяющий хранить динамический массив из элементов одного произвольного типа. [Полное описание данного контейнера на cppreference](#)

Позволяет добавлять элемент в конец, удалять с конца, обращаться по произвольному индексу.

std::set Контейнер, позволяющий хранить множество отсортированных неповторяющихся элементов произвольного типа. Реализован на КЧД. [Полное описание данного контейнера на cppreference](#)

Позволяет добавлять элементы в множество, удалять элементы из множества, а также проверять наличие определенного элемента в множестве.

std::map Контейнер, позволяющий хранить множество отсортированных пар ключ-значение. Множество отсортировано по ключу. Одному ключу соответствует одно значение. Реализован на КЧД. [Полное описание данного контейнера на cppreference](#)

Позволяет обратиться по ключу к значению, проверить наличие ключа, удалить пару ключ-значение

3 Объявления, определения и области видимости

Понятие объявления и определения, разница между ними.

Какие сущности можно объявлять в C++? Правило одного определения (ODR) для функций и переменных. Понятие области видимости (scope) и времени жизни (lifetime). Какие области видимости бывают?

Declaration / Definition. *Объявление* — это выражение, которое добавляет и опционально инициализирует новый идентификатор. *Определение* — это выражение, которое (возможно создает) и инициализирует идентификатор. Можно заметить, что всякое определение является объявлением, но обратное неверно.

В C++ можно объявлять функции и классы. Переменные можно только определять (на самом деле, используя ключевое слово `extern` можно объявить переменную, не определяя её). Сущность может быть объявлена сколь угодно много раз, но определена строго единожды (это называется One Definition Rule, сокращённо ODR).

Область видимости. Все переменные имеют определенное время жизни (lifetime) и область видимости (scope). Время жизни начинается с момента определения переменной и длится до ее уничтожения. Область видимости представляет часть программы, в пределах которой можно использовать объект. Как правило, область видимости ограничивается блоком кода, который заключается в фигурные скобки. В зависимости от области видимости создаваемые объекты могут быть глобальными или локальными.

Глобальные

Глобальные переменные определены в файле программы вне какой-либо функции или класса и могут использоваться любой функцией. Глобальные переменные существуют в течение всей жизни программы и уничтожаются лишь с завершением программы. Пример:

```
1 #include <iostream>
2
3 int n{5};    //Global variable
4
5 void print()
6 {
7     n++;
8     std::cout << "n=" << n << std::endl;
9 }
10
11 int main()
12 {
13     print();           // n=6
14     n++;
15     std::cout << "n=" << n << std::endl;    // n=7
16 }
```

Локальные

Объекты, которые создаются внутри блока кода (как правило заключенного в фигурные скобки), называются локальными. Такие объекты доступны в пределах только того блока кода, в котором они определены.

```
1 #include <iostream>
2
3 int main()
4 {
5     int n1 {2}; // scope for n1 = main
6
7     {
8         int n2 {5}; // scope = {}
9         std::cout << "n2=" << n2 << std::endl;
10        n1++; // n1 is available as it is in the scope above
11    } // n2 is dead (
12
13
14    // std::cout << "n2=" << n2 << std::endl;
15    // this would be an error because n2 died
16
17
18    std::cout << "n1=" << n1 << std::endl;
19 }
```

Локальные объекты, определенные внутри одного контекста, могут скрывать объекты с тем же именем, определенные во внешнем контексте. То есть из-за того, что переменная во вложенном блоке имеет то же имя, что и переменная из внешней области видимости, ко второй обратиться по этому имени не получится, так как внутренняя имеет приоритет, и по этому имени мы обратимся к внутренней. Это называется Variable shadowing. В некоторых случаях эта проблема решается, так как можно явно указать, как переменной мы обращаемся. Два примера таких ситуаций:

- к глобальной переменной можно обратиться, написав перед именем "::" (например ::x += 5;)
- локальная переменная метода затеняет поле класса. Тогда к полю класса можно обратиться, написав перед именем "this->" (например this->x += 5;)

4 Выражения (expressions) и операторы

Стандартные операторы и их действие над примити. Логические операторы, особенности их работы. Инкремент и декремент, отличие префиксной версии от постфиксной. Оператор присваивания и операторы составного присваивания. Наивное понятие lvalue и rvalue, требования операторов к видам value. Тернарный оператор, особенности его работы в случае разных типов или разных видов value у операндов. Оператор “запятая”. Оператор sizeof. Приоритеты операторов (помнить все не нужно, но надо знать хотя бы несколько). Ассоциативность операторов.

Выражение — это последовательность операторов и их операндов. Самыми часто используемыми операторами являются:

Common operators						
assignment	increment decrement	arithmetic	logical	comparison	member access	other
<pre>a = b a += b a -= b a *= b a /= b a %= b a &= b a = b a ^= b a <<= b a >>= b</pre>	<pre>++a --a a++ a--</pre>	<pre>+a -a a + b a - b a * b a / b a % b ~a a & b a b a ^ b a << b a >> b</pre>	<pre>!a a && b a b</pre>	<pre>a == b a != b a < b a > b a <= b a >= b a <=> b</pre>	<pre>a[...]</pre> <pre>*a</pre> <pre>&a</pre> <pre>a->b</pre> <pre>a.b</pre> <pre>a->*b</pre> <pre>a.*b</pre>	<pre>function call a(...)</pre> <pre>comma a, b</pre> <pre>conditional a ? b : c</pre>

Кроме них существуют также операторы приведения типа: `const_cast`, `static_cast` и тд. Операторы выделения памяти `new`, `delete`, `new[]`. И другие.

Особенности логических операторов Особенностью стандартных операторов `&&` и `||` является то, что второй операнд не вычисляется, если после вычисления первого, уже известно значение выражения. Например:

```
1 bool a = false;
2 bool b = true;
3
4 bool d = (a && (b = false));
5
6 std::cout << b; // -> true
```

В данном случае так как `a = false`, то при любом значении второго операнда, результат будет `false`. Поэтому выражение `(b = false)` не вычисляется и программа выведет `true`.

Стоит отметить, что такое правило работает только для логических операторов `&&` и `||`, но не для побитовых логических операторов `&` и `|`.

Определение lvalue и rvalue Любое выражение на языке C++ характеризуется двумя свойствами: типом и value category (категорией значения)

Будем (пока) говорить, что lvalue — выражения, значения которых могут стоять слева от оператора присваивания, rvalue — все остальные.

Ниже, на примере инкремента, можно будет увидеть разницу между видами value

Инкремент и декремент Стандартный оператор инкрементирования эквивалентен выражению:

```
1 a++;
2 //is equivalent to
3 a = a + 1;
```

Однако существует два оператора инкрементирования: `x++` и `++x`. Отличие заключается в том, что `++x` возвращает ссылку на **новое** значение `x` (после инкремента), выражение `++x` является lvalue, а `x++` возвращает копию **старого** значения `x` (до инкремента), выражение `x++` является rvalue. При этом инкремент нельзя применить к rvalue. Пример:

```
1 int a = 0;
2 (++a)++; // OK: a = 2. As ++a -> lvalue, it can be incremented
3 a = 0;
4 ++(a++); // NO BUENO (CE). As a++ -> rvalue, it can not be incremented.
5 ++a++; // also NO BUENO, as postfix operators have higher priority than prefix. So
        it is equivalent to ++(a++)
```

Операторы присваивания Стандартный оператор присваивания заменяет значение левого операнда на копию значения правого операнда. (для классов, если справа rvalue и у класса определён move-assignment operator, то вызывается он, иначе вызывается copy-assignment operator) и возвращает ссылку на левый операнд, где лежит новое значение, при этом выражение является lvalue. Пример:

```
1 int a;
2 int b;
3 int c = 2;
4 a = b = c = 2; // a = 2, b = 2, c = 2
```

Данное выражение называют составным присваиванием.

Оператор присваивания право-ассоциативен, то есть выражение начинает парситься справа. Сначала выполнится `c = 2`, потом `b = (c = 2)` и т.д. Пример:

```
1 int a = 2;
2 a += a += a += a; // same as 'a += (a += (a += a));'
3
4 // a += (a += (a += 2))
5 // a += (a += 4)
6 // a += 8
7
8 // a = 16
```

То же самое можно сказать и про другие операторы присваивания. Пример:

```
1 int a = 4;
2 int b = 2;
3 int c = 1;
4 a *= b *= c *= 2; // a = 16, b = 4, c = 2
```

Тернарный оператор или conditional operator. Имеет вид:

```
1 expression_1 ? expression_2 : expression_3;
```

Сначала выполняется выражение `expression_1` и приводится к `bool`. Если полученное значение — `true`, то выполняется `expression_2` и возвращается его значение. В противном случае выполняется `expression_3` и возвращается его значение.

Если `expression_2` И `expression_3` являются lvalue, то тернарный оператор вернет lvalue. В любом другом случае - rvalue;

```
1 int main() {
2     bool b = false;
3     int x, y;
4     (b ? x : y) = 1; // OK
5     (b ? x : y + 1) = 2; // CE: lvalue required as left operand of assignment
6 }
```

Operator sizeof() Унарный оператор, который возвращает размер в памяти, занимаемый операндом. Операндом может быть как некоторый объект, так и тип. Например:

```
1 int a = 2;
2 sizeof(a); // -> 4
3 sizeof(char); // -> 1
```

Operator coma Оператор запятая. Синтаксис:

```
1 expr_1, expr_2;
```

Вычисляет сначала `expr_1`, потом `expr_2` и возвращает `expr_2`. Пример:

```
1 x = (x = 5, y = 6); // y = 6; x = 6
2
3 x = (y = 6, x = 5); // y = 6; x = 5
```

Приоритет операторов

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(type){list}	Compound literal(c99)	
2	++ --	Prefix increment and decrement ^[note 1]	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of ^[note 2]	
	_Alignof	Alignment requirement(c11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10		Bitwise OR (inclusive or)	
11	&&	Logical AND	
12		Logical OR	
13	?:	Ternary conditional ^[note 3]	Right-to-left
14 ^[note 4]	=	Simple assignment	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	&= ^= =	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

Приоритет не гарантирует порядок вычисления выражений. Например:

```
1 func(expr_1, expr_2, ...);
```

Стандартом не гарантируется, что `expr_1` вычислится раньше `expr_2`. Стоит также отметить, что запятая в данном случае не является оператором, а просто частью синтаксиса вызова функции.

5 Управляющие инструкции (control statements)

Конструкции `if...else`, `for`, `while`, `do...while`, `switch`, их синтаксис, правила работы. Инструкции `break`, `continue`, `return`, их действие.

Конструкция **if**. В круглых скобках передаём выражение типа **bool** или выражение, приводимое к типу **bool**. Далее в области видимости пишутся инструкции, если же она одна, то скобки можно не писать. Существует конструкция **else**, инструкции которой выполняются, если выражение в **if** ложно. Можно использовать **else if** для проверки сложных составных условий.

```
1  #include <iostream>
2
3  int main() {
4      if (x > 0) y++;
5      if (/* bool-expression */) {
6          // do smth
7      }
8      if (/* bool-expression */) {
9          // do smth
10     } else if (/* another-bool-expression */) {
11         // do smth
12     } else {
13         // do smth
14     }
15 }
```

Ещё в `if` можно добавить `init-statement` (опционально):

```
1  #include <iostream>
2
3  int main() {
4      if (auto it = m.find(10); it != m.end())
5          return it->second.size();
6      if (char buf[10]; std::fgets(buf, 10, stdin))
7          m[0] += buf;
8  }
```

Конструкция **switch**. Используется для проверки разных значений одной переменной, случаи рассматриваются с помощью **case**. **default** используется для рассмотрения случаев, не совпавших ни с одним из предыдущих вариантов.

```
1  #include <iostream>
2
3  int main() {
4      switch (x) {
5          case 1:
6              /* do smth */
7          case 8:
8              /* do smth */
9          case 1234:
10             /* do smth */
11         default:
12             /* do smth */
13     }
14 }
```

Замечание: **switch** работает следующим образом: если какой-то **case** выполнился, то считается, что выполнены и все после него. То есть если в нашем примере $x = 1$, то выполнятся также и все случаи после **case 1**. Чтобы выполнялась только секция **case 1** нужно писать управляющее слово **break**.

```

1      #include <iostream>
2
3      int main() {
4          switch (x) {
5              case 1:
6                  /* do smth */
7                  break;
8              case 8:
9                  /* do smth */
10                 break;
11             case 1234:
12                 /* do smth */
13                 break;
14             default:
15                 /* do smth */
16             }
17     }

```

Цикл **while**. Проверяется истинно ли выражение в круглых скобках, если оно истинно, то выполняются инструкции в фигурных скобках, если же ложно, то цикл заканчивается и выполняются выражения после фигурных скобок. Как и в случае с **if**, если инструкция одна, то можно писать без фигурных скобок.

```

1      #include <iostream>
2
3      int main() {
4          while (/* bool-expression */)
5              x++;
6          while (/* bool-expression */) {
7              // do smth
8          }
9      }

```

Конструкция **do while**. Выполняются инструкции из фигурных скобок, потом проверяется условие в круглых скобках, если оно верно, то снова выполняется то, что в фигурных скобках, если же нет, то цикл заканчивается. Отличие от обычного **while** состоит в том, что если изначальное условие ложно, то в **while** цикл не выполнится ни разу, а в **do while** он сначала выполнится, а только потом произойдёт проверка.

```

1      #include <iostream>
2
3      int main() {
4          do ++x;
5          while (/* bool-expression */);
6          do {
7              // do smth
8          } while (/* bool-expression */);
9      }

```

И самый мощный цикл - **for**. Все остальные циклы можно выразить через него.

Синтаксис:

```

1      for (initializer; bool-expression; iteration) {
2          // do smth
3      }

```

initializer - некоторое объявление или выражение, которое будет проделано в самом начале цикла. Если это переменная, то её областью видимости будет тело цикла.

bool-expression - любой expression, приводимый к **bool** - условие цикла, цикл работает, пока это выражение истинно.

iteration - любой expression, обычно используется присваивание или инкремент, выполняется в

конце цикла.

Примеры:

```

1      #include <iostream>
2
3      int main() {
4          for (int i = 1; i <= n; ++i) {
5              // do smth
6          }
7          for (i = 5, j = 10; i < 10; a[i++] = i) {
8              // do smth
9          }
10         for (;;) {
11             // do smth
12         }
13     }

```

То есть принцип работы цикла: проверяем, что **bool-expression** это true, проходим тело цикла, выполняем expression из **iteration**, снова проверяем **bool-expression** и так далее.

Существуют два управляющих слова, которые влияют на ход выполнения цикла: **break** и **continue**. Слово **break** внутри цикла означает, что нужно выйти из цикла независимо от того, какие были условия цикла. Обычно **break** пишут под **if**. **break** не позволяет выйти из нескольких циклов одновременно в случае вложенности.

```

1      #include <iostream>
2
3      int main() {
4          for (;;) {
5              if (x > 0) {
6                  break;
7              }
8              // do smth
9          }
10     }

```

Слово **continue** говорит, что нужно проигнорировать все оставшиеся команды в цикле и вернуться в его начало. Тоже обычно пишется под **if**.

```

1      #include <iostream>
2
3      int main() {
4          for (int i = 0; i < 12; ++i) {
5              if (i % 5 == 0) {
6                  continue;
7              }
8              // do smth
9          }
10     }

```

Аналогично можно писать в циклах **while** и **do while**. **goto** - слово с плохой репутацией. В каком-то месте кода можно поставить **label** и тогда при использовании **goto label** программа перейдёт на строчку кода, на которой был **label**. Сейчас рекомендуется не использовать. Если с помощью **goto** попытаться запрыгнуть в какую-то область видимости, вероятно будет UB, так как нельзя использовать **goto**, минуя объявления переменных, однако выходить из циклов таким образом можно, но всё равно не рекомендуется.

```

1      #include <iostream>
2
3      int main() {
4          label:

```

```
5         for (int i = 0; i < 12; ++i) {
6             if (i % 5 == 0) {
7                 goto abcdef;
8             }
9             // do smth
10        }
11    abcdef:
12        // smth
13        if (/* bool-expression */) {
14            goto label;
15        }
16    }
```

return - способ вернуться из функции. Если функция возвращает что-то, то после **return** нужно указать что мы возвращаем. В примере при выполнении условия $x \neq y$ внутри `f()` прервутся сразу оба цикла **while**.

```
1    #include <iostream>
2    int f() {
3        while (x > 0) {
4            while (y > 0) {
5                if (x != y) {
6                    return 1;
7                }
8            }
9        }
10    }
11
12    int main() {
13        // smth
14    }
```

6 Понятия CE, RE и UB

Понятие ошибки компиляции (CE). Лексический парсер, ‘жадный’ принцип, пример лексической ошибки. Синтаксический парсер, пример синтаксической ошибки. Примеры семантических ошибок. Ошибки времени выполнения (RE). Примеры RE. Понятие undefined behaviour (UB), примеры UB.

CE будет со словами "ты чо?" © Илья Мещерин

CE — compile error. Ошибка, возникающая, когда не удастся создать исполняющий файл программы. Ошибки компиляции бывают трех видов: лексические, синтаксические и семантические.

Лексические ошибки. Лексический парсер должен разбить код на токены (отдельные значимые слова), т.е. программа представляется последовательностью операторов, ключевых слов, идентификаторов, специальных символов. Например, можно написать некорректный символ. Лексический парсер руководствуется «жадным» принципом. То есть исходя из контекста, он захватывает символы в одну лексему жадным образом (если не понятно что значит из контекста — смотри шаблоны. То есть мы можем распарсить >> как оператор битового сдвига, а можем как «закрытие» шаблона дважды).

Синтаксические ошибки. По последовательности токенов, построенной лексическим парсером, компилятору необходимо понять к какой категории относится каждый токен. Также, для выражений стоит дерево синтаксического разбора согласно таблице приоритета операций.

Семантические парсеры. После синтаксического парсинга, семантический парсер проверяет возможность выполнения операций. Например, использование необъявленного оператора, неоднозначность выполнения операции.

```

1      #include <iostream>
2
3      int main() {
4          \; // example of lexical error
5
6          6abcde; // not lexical, but semantic error.
7
8          int x;
9          std::cout << x + ; // syntax error
10
11         "abc" + 5.0f; // semantic error
12     }
```

В первой ошибке компилятор пытается понять почему \ идет вне чего-либо и не понимает, так как \ не является управляющей конструкцией, специальным символом или чем-то, с чего может начинаться выражение. Во второй ошибке компилятор распознает abcde как литеральный суффикс, но не может найти его среди известных ему, поэтому семантический парсер кидает ошибку. В третьем случае компилятор не может распарсить выражение, так как не хватает второго операнда. В последней ошибке, синтаксический парсер говорит, что все хорошо, ведь есть два операнда, но семантический парсер указывает на ошибку, так как невозможно сложить два операнда такого вида.

Даже если код запустился, он может упасть во время выполнения — **RE(runtime error)**.

Самая частая ошибка времени выполнения — Segmentation fault (segfault). Вызывается, когда происходит обращение к памяти, к которой мы не имеем права обращаться. При этом, важно уточнить, что обращение не к своей памяти — UB(про который мы поговорим позже), поэтому гарантии, что будет ошибка нет. Также частая ошибка — Floating point exception (FPE). Частое проявление — деление на 0. При этом, деление именно на целого типа на целочисленный 0, так как если выражение имеет тип float или double, то итоговое значение тоже будет иметь тип float или

double, а само значение будет inf и ошибки не будет. Третья частая ошибка — aborted, аварийное завершение программы. Ошибка с повторной очисткой памяти (double free or corruption) также относится к aborted.

Примеры вызова данных ошибок:

```

1  #include <iostream>
2  #include <vector>
3  int main() {
4  std::vector<int> v(10);
5      v[50'000] = 1; // UB, but the most possibly segfault
6
7      std::cout << 5 / 0; // FPE
8      std::cout << 5.0 / 0; // ok
9
10     v.at(100) = 1; // aborted
11 }
```

Примечание техающего: я потыкалась с этими ошибками на своем ноутбуке и код не упал на FPE, но написал, что деление на 0 — UB. Вероятно, некоторые ошибки зависят от компилятора.

Ехал UB через UB: видит UB в реке UB. Сунул UB в UB UB. UB UB UB UB © Илья Мещерин

Также в коде могут быть случаи, когда поведение неопределённое (**Undefined behaviour** — **UB**). Компилятор не может гарантировать как исполнится код (может упасть, может корректно выполнить, может еще как-то выполнить). Примеры:

```

1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> v(10);
6      v[10] = 1; // UB
7
8      int x;
9      std::cout << x; // UB
10
11     x++ + ++x; // UB
12 }
```

Идейно, программа будет пытаться вести себя так, как она вела бы себя, если бы входные данные были бы правильными. В корректных программах на с++ не должно быть строк, которые могут вызвать UB, например, таких, как в примере. Важно заметить, что компилятор или оптимизаторы могут предполагать, что UB не происходит, и оптимизировать какие-то моменты в коде, например, проверки на продолжение цикла или проверка в if. Пример такого UB:

```

1  #include <iostream>
2  int main() {
3      for (int i = 0; i < 300; ++i) {
4          std::cout << i << ' ' << i * 12345678 << '\n';
5      }
6  }
```

В данном примере, если компилировать с флагом -O2, цикл будет бесконечным, так как компилятор считает, что $i \cdot 12345678$ — не переполняется, а значит i не может быть больше 300 и вечно выполняет цикл.

Еще примеры UB: переполнение знакового типа:

[Статья на хабре про UB, к которой обращался Мещерин.](#)

[Статья на cppreference про UB](#)

7 Указатели

Операция взятия адреса. Разыменование. Арифметические операции над указателями. Особенности вывода указателей в `cout`, `sizeof` от указателей. Тип `void*` и его особенности. `nullptr` и его отличие от `NULL`. Реализация функции `swap` с использованием указателей. Примеры UB из-за неправильного использования указателей.

Оператор `&` — оператор взятия адреса. В первом приближении, все переменные просто лежат в оперативной памяти. Если вывести адрес переменной в памяти, то мы увидим какое-то 16-ричное число. Адрес имеет тип **Туре***. При этом, **Туре*** — указатель на переменную типа **Туре**. Существует обратная операция к взятию адреса — разыменование, с помощью которой мы можем посмотреть, что лежит по данному адресу.

```

1  #include <iostream>
2
3  int main() {
4      int x = 5;
5      std::cout << &x << '\n'; // out: 0x16f3d358c address
6      int* p = &x; // pointer
7      std::cout << *p << '\n'; // out: 5 dereference
8  }
```

Арифметические операции с указателями. Приведем примеры, чтобы потом их описать.

```

1  #include <iostream>
2  int main() {
3      int x = 5;
4      int* p = &x;
5
6      p + 1 // 1
7      p - 2 // 2
8      int y = 6;
9      int* t = &y;
10     t - p // 3
11 }
```

Разберем первые две строки. Здесь мы изменяем значение указателя на целое число. При таком действии, указатель сдвигается в памяти на $n \cdot \text{sizeof}(T)$ (где n — число, которое мы прибавляем или вычитаем, а T — тип значения, лежащего под указателем). Пример:

```

1  #include <iostream>
2  int main() {
3      int x = 2;
4      int* ptr = &x;
5      std::cout << ptr << ' ' << ptr + 1; // out: 0x16d41b58c 0x16d41b590
6  }
```

Мы видим, что адреса различаются на 4, т.е. `sizeof(int)`.

Размер указателя зависит от операционной системы и не зависит от того, на что он указывает. В среднем сейчас размер указателя равен 8 байтам, однако может варьироваться от 4 до 8 байт.

В третьей строке мы рассматриваем разность двух указателей. Находить разность между указателями мы можем только если они указывают на один тип. Значение, которое вернут, будет равно числу, которое необходимо прибавить ко второму указателю, чтобы получить первый. Примеры:

```

1  #include <iostream>
2  int main() {
3      int x = 2;
4      int* ptr = &x;
```

```
5      long long y = 3;
6      long long* ptr2 = &y;
7      std::cout << ptr2 - ptr; // CE
8      int* ptr3 = ptr + 3;
9      std::cout << ptr3 - ptr; // out: 3
10     }
11 }
```

Важные факты: `*ptr` — lvalue. `&x` — rvalue. При этом взятие адреса возможно только от lvalue, то есть `&(x + 1)` — CE.

Смешной пример от Мещерина из лекции:

```

1  #include <iostream>
2
3  int main() {
4      int a = 1;
5      int* p = &a;
6      {
7          int b = 2;
8          p = &b;
9      }
10     std::cout << p << '\n';
11     std::cout << *p; // UB (1)
12
13     int c = 3, d = 4, e = 5, f = 6;
14     std::cout << &c << ' ' << &d << ' ' << &e << ' ' << &f << '\n';
15     ++*p; // still UB, and possibly for one of c, d, e, f to be changed
16
17     std::cout << c << d << e << f; // may be smth different from 3456
18     std::cout << *p; // may be not 2
19 }
```

Что тут происходит? Мы создаем отдельную область видимости, где создаем переменную `b`. Возможно, наш компилятор оптимизирует это действие (можно это сделать явно через флаг `-O2`) и переменная не будет создана, а `p` присвоится произвольный адрес. После мы объявляем еще 4 переменных и есть вероятность, что одна из них будет иметь адрес, совпадающий с тем, что лежит сейчас в `p`.

Еще один пример UB с указателями:

```

1  int main() {
2      int x = 3;
3      int* p = &x;
4      ++*p; // UB
5  }
```

Мы не знаем что лежит по адресу, на который мы двинулись и разыменование — UB.

`void*` позволяет хранить указатель на что-то, не привязываясь к какому-то типу. Но при этом, `void*` нельзя разыменовывать и частично нельзя работать с адресной арифметикой. Мы не можем искать разность между указателями, но можем двигать их на целое число (см. пример). При этом можно приводить к любому другому типу указателя и работать с приведенным указателем как с каким-то обычным указателем. Важно заметить, что `void*` — не object type (incomplete type). Соответственно мы не можем создавать через `new` указатели `void*`, а также удалять их.

```

1  #include <iostream>
2
3  int main() {
4
5      int x = 1;
6      int y = 0;
7      void* ptr = &x;
8      void* ptr2 = &y;
9
10     std::cout << *ptr; // CE
11     std::cout << ptr << ' ' << ptr + 3; // OK, out : 0x7ffcacf57070 0
12     x7ffcacf57073
13     std::cout << ptr2 - ptr; // CE
14 }
```

Примечание техающего: видимо, не все компиляторы поддерживают указательную арифметику для `void*`, так как у меня Clang кидает CE на `ptr + 3`, в то время как у некоторых моих коллег

оно работает. Изучив стандарт языка C++ и используя знания, данные мне кафедрой ДИЯ, я пришла к выводу, что указательная арифметика для `void*` — UB.

Интересный факт: указатели на члены класса не могут конвертироваться в `void*`.

`nullptr` — ключевое слово, обозначающее 0 типа указателя. Имеет тип `nullptr_t`, который неявно конвертируется к любому указателю

Реализация `swap` через указатели

```
1  void swap(int* x, int* y) {  
2      int t = *x;  
3      *x = *y;  
4      *y = t;  
5  }
```

8 Виды памяти

Размещение исполняемой программы в памяти: секции `data`, `text` и `stack`. Понятие автоматической памяти (стека). Понятие `stack overflow`, пример его возникновения. Статическая память, ее особенности. Локальные статические переменные. Динамическая память. Оператор `new`, его использование в стандартной форме. Оператор `delete`, его правильное использование. Проблема утечек памяти. Проблема двойного удаления.

Память бывает статическая(`data`), динамическая и автоматическая(`stack`).

data — информация, которая хранится во время всего выполнения программы: глобальные переменные, статические переменные. Размер определяется во время компиляции, так как все переменные, которые лежат в `data`, известны при компиляции

Мы можем создавать статические переменные в локальных областях видимости: в функциях, в классах. При этом, для функции это будет один объект: то есть инициализация начальным значением будет только при первом достижении этой строки в коде:

```
1  #include <iostream>
2
3  void f() {
4      static int cnt = 0;
5      cnt++;
6      std::cout << cnt << '\n';
7  }
8
9  int main() {
10     f(); // out: 1
11     f(); // out: 2
12     f(); // out: 3
13 }
```

Аналогично, в классах инициализация будет при создании **первого объекта этого класса**.

text — бинарный код программы, тоже известен при компиляции.

stack — локальные переменные. Переменные объявленные в `main`, в вызовах функций (в том числе и аргументы функций, так как функция — отдельная область видимости. Также на стек кладется адрес инструкции, которая выполняется после выхода из функции. Размер стека — некоторая константа, изначальная задающаяся операционкой. Обычно равен 8Мб.

Stack overflow — ошибка, которая вызывается при переполнении стека. Вызовем его:

```
1  #include <iostream>
2
3  void f(int x) {
4      std::cout << x++ << '\n';
5      f(x);
6  }
7
8  int main() {
9      f(0);
10 }
```

При выполнении этого кода мы получим `segfault` и примерно 100000 вызовов рекурсии. Если мы не будем хранить переменную, то будет такая же ошибка, но вызовов будет чуть больше, так как надо хранить адрес возврата.

При этом, `segfault` вызывается не сразу. Какое-то число операций может быть совершено после того, как стек переполнился, но так как мы обращаемся не к своей памяти, на какой-то из итераций `segfault` и случится.

Также есть динамическая память, которая выделяется и очищается в runtime. Для выделения памяти существует оператор `new`. Пример выделения указателя в динамической памяти и очищение этой памяти. `new` возвращает указатель на первый байт, выделенный вам.

```
1  int main() {
2      int* p = new int; // 1
3      int* ptr = new int(1); // 2
4      delete p;
5  }
```

В строке 1 мы выделяем память под 1 объект типа `int` и конструируем его при помощи конструктора по умолчанию. В строке два мы делаем аналогичное действие, но при помощи конструктора, принимающего значение/константную ссылку/rvalue. Это верно и для указателей на произвольные типы.

Также мы можем выделять массивы в динамической памяти. И так как мы не ограничены размером стека, мы можем выделять массивы большего размера,

```
1  int main() {
2      int* arr = new int[10000];
3
4      delete [] p;
5  }
```

Важно отметить, что `delete[]` от указателя на один объект и `delete` от указателя на начало массива — UB. Еще один фанфакт: `delete nullptr` — корректное действие, которое не вызовет ошибку.

В C++, в отличие от некоторых других языков, отсутствует очищение выделенной динамической памяти при выходе из области видимости. То есть если мы выделили какую-то память через `new`, то мы обязаны вызвать `delete` для этой памяти. Но отсутствие `garbage collector`'а (сборщика мусора) ускоряет выполнение кода. Пример утечки памяти и адского сжирания оперативки:

```
1
2     void f() {
3         int* p = new int(5);
4     }
5
6     int main() {
7         while (true) {
8             f();
9         }
10    }
```

Вызов `delete` не от «нужного» нам указателя — формально UB, но чаще всего `segfault` или `abort`. `delete` от уже удаленного указателя — `aborted`, `double free` or `corruption`.

9 Массивы

Операция `[]` (квадратные скобки), ее действие. Array to pointer conversion. Сходства и различия между массивами и указателями. Оператор `new[]` и его отличие от `new`. Оператор `delete[]` и его отличие от `delete`.

Пример инициализации массивов:

```
1  int main() {
2      double a[5]; // 1
3      int b[10] = {}; // 2
4      long long c[3] = {1, 2, 3}; // 3;
5      unsigned int d[10] = {1, 2, 3}; // 4
6      unsigned int e[10]{1, 2, 3};
7  }
```

В строке 1 мы создаем массив `double` на 5 элементов. При этом значение элементов произвольное. Во строке 2 мы создаем массив `int` и заполняем его нулями. В третьей строке мы явно перечисляем элементы массива. В 4 строке мы явно перечисляем первые три элемента массива, а оставшиеся становятся нулями, как и в пятой. Обе записи корректны и массивы `d` и `e` равны(поэлементно)

Индексация массива идет с нуля. Выход за границы массива — UB. Выход за границы массива слишком далеко, скорее всего, вызовет `segfault`. Массивы хранятся на стеке. Соответственно, если размер массива превышает размер стека, то будет `segfault` из-за `stack overflow`.

```
1  int main() {
2      int a[10]; // create an array
3      a[10] // UB, maybe without segfault
4      a[100000]; // possibly segfault
5      a[-1]; // UB
6
7      int b[100000000]; // segfault
8      static int c[100000000]; // OK, because in data
9  }
```

С массивами можно работать как с указателями, так как мы можем неявно конвертировать массив в указатель на начало. То есть для них работает указательная арифметика

```
1  int main() {
2      int a[5] = {1, 2, 3, 4, 5};
3      *(a + 2); // OK
4
5      a[2] == *(a + 2);
6
7      int *p = a + 3; // array to pointer conversion
8      p[-1]; // OK
9
10     a[2] == 2[a]; // OK, because *(a + 2) == *(2 + a)
11 }
```

При этом, массивы нельзя присваивать друг другу. Инкрементировать массивы тоже нельзя. Также `sizeof` для указателей и массивов работает по-разному.

```
1  #include <iostream>
2  int main() {
3      int a[5] = {1, 2, 3, 4, 5};
4      int b[5]{};
5      a = b; // CE
6      a++; // CE
7      a += 1; // CE
8      int* p = a + 3;
```

```

9      std::cout << sizeof(a) << ' ' << sizeof(p); // out: 20 8
10  }

```

Мы не можем перегружать функции на массивы и указателями. Они будут считаться одинаковыми:

```

1  void f(int *p) {
2      std::cout << "biba";
3  }
4  void f(int a[3]) { // CE
5      std::cout << "boba";
6  }

```

Массивы в динамической памяти выделяются тоже при помощи new. Несмотря на то, что мы просим выделить массив, нам возвращается указатель.

```

1  int main() {
2      int* a = new int[100];
3
4      delete[] a; // delete a -- UB
5  }

```

Отличие new от new[n]: первый оператор выделяет одну ячейку памяти под нужный тип, в то время как второй выделяет n подряд идущих ячеек памяти. Аналогично, delete удаляет только сам указатель, а delete[] удаляет весь выделенный массив. При этом размер массива передавать не нужно, он автоматически сохраняется в памяти при выделении массива.

Двумерные массивы, указатели на массивы, массивы указателей, указатели на массивы указателей, зовите санитаров

```

1  int main() {
2      int a[5][5]; // 2d array
3      int* b[5]; // array of 5 ptr to int
4      int (*c)[5]; // pointer to array of 5 ints
5  }

```

Выделение массива переменной длины:

```

1  int main() {
2      int n;
3      std::cin >> n;
4      int a[n];
5  }

```

Так делать не очень хорошо, однако такой код может компилироваться. Однако это не регламентировано стандартом, поэтому на одном компиляторе это может скомпилироваться, а на другом нет. Массивы переменной длины лучше выделять в динамической памяти:

```

1  int main() {
2      int n;
3      std::cin >> n;
4      int* arr = new int[n];
5  }

```

10 Функции

Понятие перегрузки функций. Общие правила разрешения перегрузки. Использование аргументов по умолчанию в функциях. Пример неоднозначности при перегрузке функций. Особенности передачи массивов в функции.

В коде могут существовать функции с одинаковым именем, но разным набором аргументов.

```
1 void f() { }
2 void f(int x) { }
3 int f(double x) { return 0; }
4 // everything is correct
```

При этом, в случае перегрузки, выбирается версия, подходящая под передаваемое значение.

```
1 #include <iostream>
2
3 void f(int) {
4     std::cout << 1;
5 }
6
7 void f(double) {
8     std::cout << 2;
9 }
10
11 int main() {
12     f(1);
13     std::cout << ' ';
14     f(1.0); // out: 1 2
15 }
```

При этом, в данном примере, если бы мы передавали в `f` значения типа `float`, то выбрался бы `double`, так как расширение до `double` предпочтительней урезки до `int`. Однако, если бы функция принимала бы `float`, а передавали мы `double`, то было бы СЕ (непонятно как обрезать тип)

```
1 void f(int) {}
2 void f(float) {}
3
4 int main() {
5     f(0.0); // СЕ
6 }
```

Также, СЕ будет вызывать создание двух функций, имеющих одинаковый принимаемый тип, даже если у них разные возвращаемые типы.

```
1 void f() {}
2 int f() {} // СЕ
```

Давайте строго опишем правила выбора функций компилятором для перегрузки: Перегружаемые функции имеют одинаковое имя, но разное количество или типы аргументов. Тип должен однозначно выбираться в `compile-time`, так как компилятор проставляет адреса в машинном коде для прыжков в функцию. Предпочтения в выборе перегруженной функции отдается в таком порядке: точно соответствие, `promotion`(расширение типа), `conversion`(приведение одного типа к другому). При том, если не существует функции, которая при выборе будет более приоритетного типа, а для текущего рассматриваемого типа существует несколько подходящих функций, то будет СЕ. Пример: у нас нет точного соответствия и мы не можем расширить тип, но у нас есть несколько других типов, для которых тип, передаваемый в функцию придется сужать — СЕ.

Чтобы передавать функции в функции, существуют указатели на функции. Указатель указывает на адрес функции в машинном коде. Опишем синтаксис:

```

1     bool cmp(int x, int y) {
2         return x > y;
3     }
4
5     int main() {
6
7         bool (*p)(int, int) = &cmp; // int, int - arguments, bool - type of
returning value
8         bool (*pp)(int, int) = cmp; // function to pointer conversion
9         std::cout << (void*)p; // we also can print pointers to functions as usual
pointers
10    }

```

Указатели на функции работают даже с перегруженными функциями. Нужный адрес выбирается по типу определяемого указателя.

```

1     void f(int) {}
2     void f(double){}
3     void f(char);
4
5     int main() {
6         &f; // CE
7         void (*p)(int) = &f; // OK
8         void (*pp)(double) = &f; // OK
9         void (*p2)(char) = &f; // linker's error
10    }

```

Пара смешных примеров с лекции продвы:

```

1     int sqr(int x) {
2         return x * x;
3     }
4
5     double sqr(double x){
6         return x * x;
7     }
8
9     int main() {
10        int (*p)(int) = sqr;
11        double (*pp) (double) = (double (*)(double))(p); // C-style cast, UB
12
13
14        int (*f[5])(int); // 1
15
16        (int (*)(int))(*g[3])(int[5]); // 2
17    }

```

1 — массив из 5 указателей на функцию, принимающую int и возвращающую int.

2 — массив из 3 указателей на функцию, принимающую массив из 5ти int'ов и возвращающую указатель на функцию, принимающую int и возвращающую int

При передаче массивов в функции, массив из указателей и указатель на указатель воспринимаются одинаково. Поэтому 1 и 3 строки вызовут СЕ, так как переопределение функции. В то время как в строке 2 объявлен указатель на массив, который будет верно восприниматься.

```

1     void f(int**) {} // 1
2     void f(int(*)) {} // 2
3     void f(int*[5]) {} // 3

```

В функции можно передавать аргументы по умолчанию. При этом они всегда ставятся в конце перечисления аргументов. Добавление аргументов по умолчанию не меняет сигнатуры функций. Пример:


```
1 void f(int x) {}  
2 void f(int y, int t = 0) {} // CE
```

В [этой](#) статье на [crrreference](#) много примеров, как можно использовать и задавать значения по умолчанию. Так как примеров достаточно много, советуем почитать статью.

11 Ссылки

Дилемма с присваиванием. Синтаксис ссылок. Интуитивный смысл. Особенности инициализации и присваивания ссылок. `sizeof` ссылки и адрес от ссылки. Неоднозначность между `int` и `int&`. Что происходит при окончании жизни ссылки? Реализация `swap` через ссылки. Можно ли создать ссылку на ссылку, указатель на ссылку, ссылку на указатель, массив ссылок, ссылку на массив? Особенности возврата ссылок из функций. Проблема висячих ссылок. Как хранятся ссылки в памяти?

Одна из ключевых концепций, которая появилась в C++ в сравнении с C, — ссылки (references).

К примеру

```
1 int main() {
2     std::string a = "Perecdacha";
3     std::string b = a;
4 }
```

Объекты `a` и `b` не связаны (это 2 абсолютно разных объекта). Для того, чтобы сказать, что `b` не новый объект, а новое имя для старого объекта, используются *ссылки*.

Примечание: в случае Python или Java это не так, но в C++ в Run time нет «сборщика мусора».

```
1 int main() {
2     std::string a = "Komsa";
3     std::string& b = a;
4 }
```

Теперь `a` и `b` — один и тот же объект (при изменении `a` меняется `b`, при изменении `b` — `a`).

В следующем коде ссылка перестаёт существовать при выходе из области видимости, но объект все ещё есть:

```
1 int main() {
2     std::string a = "abdfsdtewrffgfasra";
3     {
4         std::string& b = a;
5         b[0] = 'b'; //now a[0] = 'b'
6     }
7 }
```

Интуиция про ссылки: ссылка это другое название исходного объекта, они неотличимы (за исключением кое-чего, о чем мы пока умолчим).

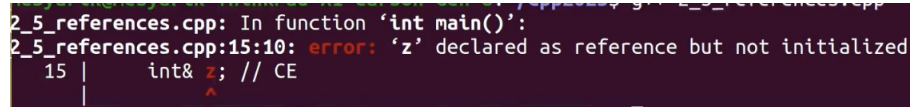
```
1 int main() {
2     std::string a = "abdfsdtewrffgfasra";
3     std::string& b = a;
4
5     std::string c = "sfdafdfa";
6     b = c; //now a = c
7     &b; //equiv &a
8     sizeof(b); //equiv sizeof(a)
9 }
```

Можно привести такой пример с ссылками, как: МФТИ и Физтех, — это два способа «сказать» про одно и то же, то есть «я поступил в МФТИ» $\equiv$ «я поступил на Физтех».

Ссылки позволяют работать с исходными переменными при передаче их в функцию.

Рассмотрим следующий код:

```
1 void swap(int& a, int& b) {
2     int t = a;
3     a = b;
4     b = t;
5 }
6
7 int main() {
8     int x = 1;
9     int y = 2;
10    int& z; //CE, because we must initialize references
11
12    swap(x, y); //now x = 2, y = 1
13 }
```



```
2_5_references.cpp: In function 'int main()':
2_5_references.cpp:15:10: error: 'z' declared as reference but not initialized
   15 |     int& z; // CE
      |         ^
```

Также стоит сказать, что нет ссылок на ссылки (как и указателей на ссылки), и если мы попытаемся так сделать, это будет CE:

```
1 int main() {
2     int x = 1;
3     int& y = x;
4     int& & z1 = y; //CE
5     int&* z2 = &y; //CE
6     int& z3 = y; //OK
7 }
```

Но ссылку на указатель можно сделать.

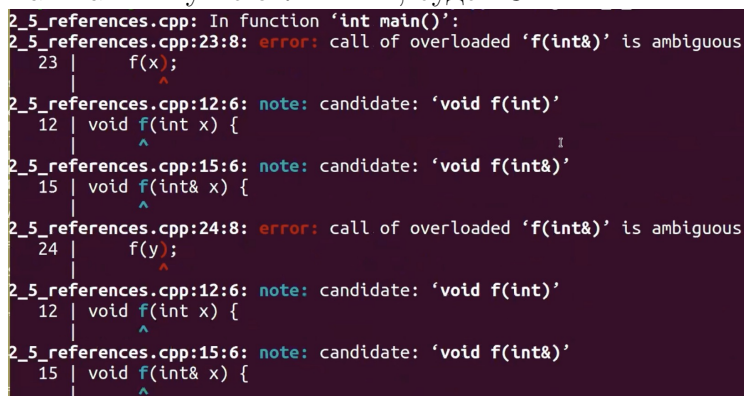
Фан-факт: нельзя объявить вектор ссылок (из-за того, что указатели на ссылки запрещены), как и массив ссылок, то есть

```
1 std::vector<int&> v; //CE
2 int& a[10]; //CE
```

Рассмотрим следующий код:

```
1 void f(int x) {}
2 void f(int& x) {}
3
4 int main() {
5     int x = 1;
6     int& y = x;
7     f(x); //CE
8     f(y); //CE
9 }
```

Так как x и y не отличимы, будет CE:



```
2_5_references.cpp: In function 'int main()':
2_5_references.cpp:23:8: error: call of overloaded 'f(int&)' is ambiguous
   23 |     f(x);
      |     ^
2_5_references.cpp:12:6: note: candidate: 'void f(int)'
   12 | void f(int x) {
      |     ^
2_5_references.cpp:15:6: note: candidate: 'void f(int&)'
   15 | void f(int& x) {
      |     ^
2_5_references.cpp:24:8: error: call of overloaded 'f(int&)' is ambiguous
   24 |     f(y);
      |     ^
2_5_references.cpp:12:6: note: candidate: 'void f(int)'
   12 | void f(int x) {
      |     ^
2_5_references.cpp:15:6: note: candidate: 'void f(int&)'
   15 | void f(int& x) {
      |     ^
```

Рассмотрим следующий код:

```

1 int& f(int x) {
2     int y = ++x;
3     return y;
4 }
5
6 int main() {
7     int x = 1;
8     int& y = f(x);
9     std::cout << y;
10 }
```

С точки зрения компиляции это корректный код, но вообще это UB (так как мы возвращаем ссылку на уничтоженный объект). Называется данное явление — *висячая ссылка* (dangling reference).

Но следующий код корректен:

```

1 int& f(int& x) {
2     int& y = ++x; //or static int y = ++x;
3     return y;
4 }
5
6 int main() {
7     int x = 1;
8     int& y = f(x);
9     std::cout << y;
10    f(x) = 3; //OK, because the result of the function is lvalue
11 }
```

Если бы `f` возвращала `int`, было бы СЕ.

Замечание: следующий код, очевидно, не корректен

```

1 int main() {
2     int& x = 1; //CE
3 }
```

Так как ссылку можно инициализировать только lvalue выражением, в частности

```

1 int& y = ++x; //OK
2 int& y = x++; //CE
```

Следующий код, формально, тоже является UB:

```

1 int& f(int x) {
2     int& y = x
3     return y;
4 }
```

Так как мы возвращаем ссылку на локальную переменную, которая была уничтожена.

12 Константы

Синтаксис объявления. Интуитивный смысл. Константные указатели и указатели на константу, что можно и что нельзя. Как можно присваивать. Константные ссылки. Три вида передачи аргументов в функции: когда нужно использовать каждый из них? Как лучше принимать примитивные типы? Можно ли делать перегрузку между `int&` и `const int&`? Когда и как происходит продление жизни объектов ссылками?

В C++ есть *модификатор типа*, называющийся **const**. Пример его использования:

```
1 int main() {
2     const int c = 5;
3
4     return 0;
5 }
```

Идея в том, что переменную `c` нельзя менять, то есть не получится сделать `++c` или `c = 6` (это будет СЕ).

На самом деле, интуиция «константы — это то, что менять нельзя» не совсем правильная, корректнее мыслить константы так: константы — это тип, похожий на исходный, но у которого убрана часть операций.

Пример: $\mathbb{R}$ и $\mathbb{C}$; с вещественными и комплексными числами можно делать какие-то операции (складывать, вычитать, умножать), но у комплексных отсутствует, к примеру, сравнение (СЕ), в остальном операции определены также. То есть `const int` во всем как `int`: занимает столько же места, представляется в памяти также, арифметические операции работают также, как над `int`, просто часть операций у него не определена.

Это операции, которые потенциально могут менять эту переменную, например, инкремент, присваивание, составное присваивание то есть такой код некорректен, несмотря на то, что `c` не поменялось:

```
1 int main() {
2     const int c = 5;
3     c = 5; //would be a CE
4     c *= 1; //would be a CE
5     ++c; //would be a CE
6
7     return 0;
8 }
```

Рассмотрим следующий код:

```
1 int main() {
2     const int c = 5;
3     int& x = c;
4
5     return 0;
6 }
```

Это тоже СЕ, так как когда мы создаем ссылку на константу, `const` должен сохраняться (мы из константного объекта как бы попытались сделать не константный). То есть нужно исправить `int& x = c` на `const int& x = c`.

В «обратную» же сторону код корректный, то есть

```
1 int main() {
2     int x = 5;
3     const int& c = x; //here an implicit conversion takes place
4 }
```

Происходит неявная конверсия, то есть навешивание const под ссылку. Это стоит понимать как ссылку, которая дает лишь ограниченный доступ к int-у. Стоит отметить, что изменение x влияет на c, то есть

```
1 int main() {
2     int x = 5;
3     const int& c = x;
4     ++x; //now x = 6 and c = 6
5     std::cout << c; //output: 6
6 }
```

Итого: используя x мы можем читать и писать, а используя c только читать.

Однако так как это C++ существует способ, используя константную ссылку, поменять значение под ней (об этом чуть позже, но это «кринж»), но не напрямую.

Рассмотрим вопрос передачи аргументов в функцию. Есть несколько способов это сделать:

```
1 void f(std::string str) {} //a copy of the object would be created
2 void f(std::string& str) {} //we could change original object
3 void f(const std::string& str) {} //no copying and no changing of original object
4 void f(const std::string str) {} //cringe
```

1. Мы можем менять созданную локально строку без изменения «изначальной», но будет создана копия.
2. Используется, когда функция должна поменять исходную переменную, например, swap.
3. Используется, когда мы не хотим копировать объект (так как это может быть очень долго для нетривиальных типов), но хотим запретить его изменение, что встречается крайне часто.
4. «Кринж» и не имеет смысла, но ради порядка его нужно было упомянуть.

В случае, если мы имеем дело с тривиальным типом, по типу int или double, то нет разницы принимать по ссылке или копию (так как копирование «дешево»), если у нас нет цели изменить изначальный объект, при этом способ 3 тоже перестает иметь смысл (и будет даже тяжеловеснее).

Теперь рассмотрим следующий код:

```
1 void f(const std::string& str) {}
2 void f(const int& numb) {}
3
4 int main() {
5     f("You know what I'm saying?"); //woudn't be a CE
6     f(5); //woudn't be a CE
7
8     return 0;
9 }
```

Если бы не было const в аргументах функции, то данный код, очевидно, был бы не корректен, но в данном случае он корректен. Константные ссылки ведут себя не как обычные в контексте инициализации, их можно инициализировать gvalue, это сделано, чтобы не нужно было создавать переменную для передачи в функцию.

Следующий код тоже корректен:

```
1 int main() {
2     const std::string& str = "ABOBA"; //Lifetime extension
3
4     return 0;
5 }
```

Это называется продление времени жизни объекта, то есть данный «объект» будет существовать до тех пор, пока существует str. Но это работает только в локальной области видимости, то есть код

```
1 const int& f(int x) {
2     return x + 1;
3 }
```

Корректен с точки зрения компиляции, но является UB, так как константные ссылки продлевают жизнь объектам только лишь локально объявленным, как переменные, но не при возврате из функции.

Рассмотрим следующий код:

```
1 void f(int& x) {}
2 void f(const int& y) {}
3
4 int main() {
5     int x = 5;
6     f(x); //first version
7     const int y = 6;
8     f(y); //second version
9
10    return 0;
11 }
```

Такая перегрузка корректна. Но не будь там ссылок, код был бы некорректен, то есть

```
1 void f(int x) {}
2 void f(const int y) {}
```

Такая перегрузка некорректна.

```
1 int main() {
2     int x = 5;
3
4     const int* p = &x;
5     ++*p; //CE
6     ++p; //OK
7
8     int* const pc = &x;
9     ++*p; //OK
10    ++p; //CE
11
12    return 0;
13 }
```

Первое — указатель на константный int (при разыменовании получаем константу).

Второе — константный указатель на неконстантный int.

Также стоит отметить:

```
1 int main() {
2     int x = 5;
3     const int* p = &x;
4     int* pp = p; //would be a CE
5
6     return 0;
7 }
```

Так как мы пытаемся нарушить константность. Так же, как и в следующем коде

```
1 int main() {
2     const int x = 5;
3     const int* p = &x;
4
5     int* pp = &x; //also CE
6
7     return 0;
8 }
```

Но

```
1 int main() {
2     const int x = 5;
3     const int* const p = &x; //OK
4
5     return 0;
6 }
```

Корректно и будет константным указателем на константу.

13 Приведения типов

Разница между C-style cast и `static_cast`. Примеры корректного и некорректного использования того и другого.

В C++ есть несколько способов привести один тип к другому (явно): `static_cast`, `reinterpret_cast`, `const_cast`, C-style cast.

```

1 int main() {
2     int x = 5;
3     static_cast<double>(x); //OK
4
5     reinterpret_cast<float>&(x);
6     float& f = reinterpret_cast<float>&(x); //OK
7     std::cout << f; //would assume that x is float; formally UB
8
9     f = 3.14;
10    std::cout << x; //UB, because x is int
11
12    return 0;
13 }
```

Если нужно сделать приведение типов, но не знаем какое именно, то скорее всего, это `static_cast`.

В свою очередь, `reinterpret_cast` позволяет смотреть на биты одного типа так, как будто это другой тип. Формально это UB (почти всегда, если мы не уверены, что при конверсии биты хранились также), но не является CE.

Важный момент: `reinterpret_cast` делается только к указателю или ссылке, нельзя с помощью него сделать новый объект; по аналогии с константными ссылками на неконстантные объекты, мы не делаем новый объект, но смотрим на старый по-новому. Но даже он не позволяет «снять» константность с типа.

Приведём пример некорректного использования `static_cast` (преобразование между несвязанными типами):

```

1 class Base {};
2 class Derived : public Base {};
3 class Unrelated {};
4
5 int main() {
6     Derived* derived = new Derived();
7
8     Unrelated* unrelated = static_cast<Unrelated*>(derived); //CE
9
10    delete derived;
11    return 0;
12 }
```

Для того чтобы «снять» константность с типа существует `const_cast`, применяется крайне редко.

```
1 int main() {  
2     const int x = 5;  
3     const_cast<int&>(x) = 6; //UB  
4     int& y = const_cast<int&>(x); //UB  
5     y = 6; //Most likely x still would be 5  
6 }
```

Как и `reinterpret_cast`, `const_cast` может быть сделан только к ссылке или указателю (снимается константность из-под). Формально это UB. Не UB это, если изначально `int` был не константный, но мы получили его по константной ссылке.

C-style cast — это приведение типов, которое используется в языке C и поддерживается в C++. Оно имеет следующий синтаксис (оба корректны):

```
1 T variable = (T)expression;  
2 T variable = T(expression);
```

Стоит отметить: C-style cast не предоставляет дополнительных проверок времени компиляции, что делает его менее безопасным по сравнению с `static_cast`.

На самом деле, C-style cast совмещает возможности всех трех вышеперечисленных кастов, то есть когда мы так пишем: сначала пытается сделаться `const_cast`, если это не получилось, то `static_cast`, если и это не получилось, то `static_cast`, на который навешан `const_cast`, потом попытка `reinterpret_cast`, потом `reinterpret_cast`, на который навешан `const_cast`, и только если это не получилось, кидается СЕ.

В нашем code style он запрещён.

14 Классы и структуры, модификаторы доступа

Поля и методы. Операторы ‘точка’ и ‘стрелочка’. Инициализаторы полей по умолчанию. Синтаксис определения методов вне тела структуры. Ключевое слово `this`. Понятие инкапсуляции. Ключевые слова `private` и `public`, их действие и использование. Разница между классом и структурой. Пример ошибки из-за нарушения прав доступа. Функции-друзья и классы-друзья.

По сути программа на C++ — объявление/определение типов, операции над ними и завершение. В нём стоит выделить такую крайне популярную парадигму — ООП, мы как бы определяем свои типы данных и операции над ними. Существует несколько способов это сделать, остановимся пока на структурах и классах.

Синтаксис определения структур следующий:

```
1 struct S {
2     //Fields
3     int x;
4     char c;
5     double d;
6
7     //Methods
8     void f(int y) {
9         std::cout << x + y; //x will be taken from the fields
10    }
11 };
12
13 int main() {
14     S s;
15     s.x = 4; //now field x in object s equals 4
16     std::cout << s.x; //output: 4
17
18     s.f(5); //output: 9
19 }
```

Классы определяются аналогично и они почти во всем (за исключением двух моментов) одинаковы, поэтому все, что мы говорим для структур, верно для классов и наоборот.

В теле структуры можно объявлять переменные, функции, `using`-и, другие структуры (нельзя `expression`-ы).

Поля — то, что мы храним в структуре/классе (переменные).

Методы — функции, которые мы объявляем в структуре/классе.

Начиная с C++11 полям можно присваивать значения по умолчанию (иначе они будут проинициализированы рандомными значениями, обращение к ним было бы UB):

```
1 struct S {
2     int x = 1;
3     char c = 'a';
4     double d = 3.14;
5 };
```

Если мы не определили значение поля, то его значение будет значением по умолчанию.

Кроме точки можно использовать **оператор стрелочка** « $\rightarrow$ » для обращения к структуре, по своей сути $a \rightarrow b \equiv (*a).b$, то есть способ обратиться к полю/методу объекта, имея не сам объект, а указатель на него, к примеру

```
1 struct S {
2     int x = 1;
3 };
4
```

```

5 int main() {
6     S* p = new S;
7     p->x;
8
9     delete p;
10 }

```

Есть ключевое слово `this`, обозначающее указатель на текущий объект, к примеру

```

1 struct S {
2     int x = 1;
3
4     void f(int x) {
5         std::cout << x + x; //both x are taken from the function arguments
6         std::cout << this->x + x; //the first x is taken from the fields, the
          second one is from the arguments
7     }
8 };

```

Методы можно объявлять внутри класса, а определять снаружи:

```

1 struct S {
2     int x = 1;
3
4     void f(int);
5 };
6
7 void S::f(int y) {
8     std::cout << x + y;
9 }

```

Также при создании структуры (в случае очень простых структур) можно явно указать, чему должны быть равны её поля в порядке их объявления.

```

1 struct S {
2     int x = 1;
3     char c = 'a';
4     double d = 3.14;
5 };
6
7 int main() {
8     S s{2, 'b'}; //aggregate initialization
9 }

```

Притом в данном случае третье поле будет равно значению по умолчанию, а если мы укажем больше значений — СЕ.

Важно понимать, что

```

1 struct S {
2     int x = 1;
3
4     struct SS { //inner struct
5         int a = 0;
6         void f(int b) {
7             std::cout << x + b; //would be a CE
8         }
9     };
10 };
11
12 int main() {
13     S::SS obj;
14     std::cout << obj.a;
15 }

```

Хоть SS и объявлено внутри S, но её поля не являются полями S, так же как и поля S — не поля SS.

Также структуры можно объявлять внутри функций.

Стоит отметить, что поля структуры хранятся в памяти в порядке их объявления, то есть в нашем случае они так и будут лежать подряд: int, char, double, — а в силу требований к кратности адресов $\text{sizeof}(S) = 16$ и в целом sizeof структуры может отличаться при различных «расположениях» полей (методы не влияют на это).

Есть требование, когда мы создаем массив из n структур, его $\text{sizeof} = n \times \text{sizeof}(\text{структуры})$, поэтому в нашем случае S положится на адрес, кратный 8 (в силу double).

Если в структуре нет полей, она занимает 1 байт (для того, чтобы адреса были разные).

В ООП обычно выделяют 3 популярных принципа: *инкапсуляция*, *наследование* и *полиморфизм*. Строгого определения инкапсуляция так и не сформировалось, одно из пониманий инкапсуляции: поля и методы (данные) хранятся как бы в одной структуре (капсуле), — другое: должно быть разграничение доступа, то есть к данным напрямую имеют доступ только методы работы с этими данными, а «внешние» имеют доступ только к методам, но не к данным напрямую (классический пример — интерфейс стиральной машинки, кнопки — методы).

Существует **3 модификатора доступа**: public, private, protected.

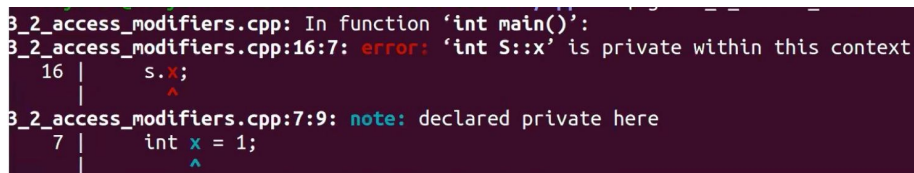
- private. Это значит не доступно никому, кроме моих методов и моих друзей.
- public. Это доступно всем
- protected. Доступно моим методам и моим друзьям, а также моим наследникам. (protected = private + доступ наследникам). Разобран подробно в 18 билете

Остановимся подробнее на первых двух: public — доступ из-вне разрешен, private — запрещен.

Пример

```

1 struct S {
2     private:
3         int x = 1;
4     public:
5         void f(int y) {
6             std::cout << x + y;
7         }
8 };
9
10 int main() {
11     S s;
12     s.x //CE
13 }
```



```

3_2_access_modifiers.cpp: In function 'int main()':
3_2_access_modifiers.cpp:16:7: error: 'int S::x' is private within this context
   16 |     s.x;
      |     ^
3_2_access_modifiers.cpp:7:9: note: declared private here
    7 |     int x = 1;
      |     ^
```

Обратим внимание: CE за нарушение доступа (то есть проверка прав доступа выполняется в compile time). Но из методов можно обращаться (даже если они определены вне класса). Никаких ограничений в плане объявлений нет, то есть public и private могут чередоваться сколько угодно раз, могут не быть вообще, могут идти в любом порядке.

Aggregate initialization перестаёт работать, если есть приватные поля.

Главное (и единственное) отличие классов от структур: если не написать модификатор доступа, то в структуре все по умолчанию public, а в классе — private. Во всем остальном они одинаковы.

Существует понятие *друзей*: можно объявить какую-нибудь функцию или класс friend-ом класса, тогда она будет иметь доступ к приватной части класса не смотря на то, что она внешняя. Синтаксис такой:

```
1 class S {  
2     private:  
3         int x = 1;  
4     public:  
5         friend void modify(S&);  
6 };  
7  
8 void modify(S& s) {  
9     ++s.x; //OK  
10 }
```

Не важно в какой части написан friend: в публичной или приватной, в начале или в конце, главное — в числе того, что объявлено в классе.

Дружба не является симметричной: ты не друг своего друга, если он об этом не сказал.

Дружба не является транзитивной: друг твоего друга — не твой друг (прописывать нужно явно).

15 Конструкторы и деструкторы

Понятие конструктора. Пример простейшего конструктора. Списки инициализации в конструкторах и их преимущество перед присваиваниями. Пример класса, для которого списки инициализации в конструкторах строго необходимы. Конструктор по умолчанию, условия его генерации компилятором. Перегрузка конструкторов. Пример ситуации, когда необходим нетривиальный конструктор. Понятие деструктора. Порядок вызова конструкторов и деструкторов в разных ситуациях. Использование слов `default` и `delete` при объявлении методов.

Существуют методы, создающие объект класса из каких-то параметров, они называются *конструкторы*.

Рассмотрим следующий пример:

```
1 class Complex {
2 private:
3     double re = 0.0;
4     double im = 0.0;
5 public:
6     Complex(double real, double imag) {
7         re = real;
8         im = imag;
9     }
10    Complex(double real) {
11        re = real;
12    }
13 }
14
15 int main() {
16     Complex c(1.0, 2.0); //OK
17     Complex c1{1.0, 2.0}; //OK, it's not aggregate initialization
18     Complex c2 = {1.0, 2.0}; //OK
19 }
```

Конструкторы можно перегружать (по правилам перегрузки функций).

Первый конструктор написан плохо, потому что мы вместо инициализации полей делаем присваивание. Когда мы заходим в тело конструктора, поля уже созданы и проинициализированы и к ним можно обращаться, как к готовым объектам (к примеру, если бы было поле `std::string`, то при входе в конструктор был бы вызван его конструктор, то есть он бы был создан и проинициализирован).

Правильнее в данном случае было бы написать:

```
1 //Member initializer list
2 Complex(double real, double imag): re(real), im(imag) {}
```

Это называется список инициализации. Он вызывает конструкторы полей, если там нетривиальные объекты. Стоит отметить, что названия могут совпадать и оно будет распаршено однозначно, то есть

```
1 Complex(double re, double im): re(re), im(im) {} //OK, but cringe
```

Поля нужно инициализировать в том порядке, в котором они объявлены. Нужно отметить, что тело конструктора, должно быть непустым, только если есть какая-то нетривиальная логика действий.

Без `initializer list` не обойтись, если полями являются константы и ссылки.

Приведем пример нетривиального конструктора на примере String:

```

1 class String {
2 private:
3     char* arr_;
4     size_t sz_;
5     size_t cap_;
6
7 public:
8     //just for an example
9     String(size_t count, char c): sz_(count), arr_(new char[sz_ + 1]), cap_(count +
10         1) { //UB!
11         .....
12     }
13
14     String(size_t count, char c): arr_(new char[count + 1]), sz_(count), cap_(count
15         + 1) {
16         memset(arr_, c, count);
17     }
18 };

```

UB из-за того, что сначала будет проинициализировано `arr_`, но `sz_` ещё формально не определен, а мы пытаемся к нему обратиться, итогом будет выделена на непонятное число байт (инициализация идёт в порядке объявления).

Деструктор — то, что вызывается в конце времени жизни объекта. Синтаксис такой:

```

1 class String {
2 private:
3     //the same as before...
4
5 public:
6     //some code
7
8     ~String() { //without arguments
9         delete[] arr_;
10    }
11 };

```

Деструкторы вызываются в обратном порядке полей.

Далеко не для всех классов нужен нетривиальный деструктор.

Вернёмся к конструкторам. Есть такое понятие: конструктор по умолчанию, то есть конструктор без аргументов. Его особенность в том, что если мы не написали никакого конструктора, как и деструктора, то будет автоматически неявно сгенерирован конструктор по умолчанию, который ничего не делает (в том смысле, что он просто проинициализирует поля по умолчанию). Но если есть явно прописанный конструктор, то компилятор уже не генерирует его (нужно дописывать самому). Синтаксис в нашем случае такой:

```

1 String(): arr_(nullptr), sz_(0), cap_(0) {}

```

Есть понятие: делегирующий конструктор (с C++11). Пример

```

1 class String {
2 private:
3     char* arr_ = nullptr;
4     size_t sz_ = 0;
5     size_t cap_ = 0;
6
7     String(size_t count): arr_(new char[count + 1]), sz_(count), cap_(count + 1) {}
8
9 public:
10    String() {}
11 };

```



```

12     String(size_t count, char c): String(count) {
13         memset(arr_, c, count);
14     }
15
16     String(const char* str): String(strlen(str)) {
17         memcpy(str, arr_, sz);
18     }
19
20     ~String() {
21         delete[] arr_;
22     }
23 };

```

В C++11 и более поздних версиях появились ключевые слова `default` и `delete`, которые помогают «управлять» специальными методами класса, такими как конструкторы, деструкторы и операторы присваивания.

Ключевое слово `default` используется для явного указания компилятору, что нужно сгенерировать реализацию метода по умолчанию. Это может быть полезно в следующих случаях:

1. Когда вы хотите явно указать, что метод должен использовать реализацию по умолчанию;
2. Когда у вас есть пользовательские конструкторы или деструкторы, но вы также хотите использовать методы по умолчанию для других специальных методов.

Пример использования `default`:

```

1 class MyClass {
2 public:
3     MyClass() = default; //default constructor
4     MyClass(const MyClass&) = default; //default copy constructor
5     MyClass& operator=(const MyClass&) = default; //default assignment operator
6     ~MyClass() = default; //default destructor
7
8     //user constructor
9     MyClass(int value) : value(value) {}
10
11 private:
12     int value;
13 };

```

Ключевое слово `delete` используется для явного запрета вызова определенного метода. Это полезно, когда вы хотите предотвратить использование некоторых особых методов, таких как копирующий конструктор или оператор присваивания.

Примером может служить `StackStorage` из задачи `List`. Также приведем следующий пример:

```

1 class NonCopyable {
2 public:
3     NonCopyable() = default;
4     NonCopyable(const NonCopyable&) = delete; //prohibition of the copy
5     constructor
6
7     //user constructor
8     NonCopyable(int value) : value(value) {}
9
10 private:
11     int value;
12 };
13
14 int main() {
15     NonCopyable obj1(10);

```

```
15     NonCopyable obj2 = obj1; //Error: copy constructor deleted
16 }
```

16 Правило трёх

Конструктор копирования, условия его генерации компилятором.

Поверхностное и глубокое копирование.

Реализация копи-конструктора и оператора присваивания на примере класса `String`.

Формулировка ‘правила трех’.

Условия генерации оператора присваивания компилятором.

В данном параграфе все удобно рассматривать на примере задачи `String`. Ее внутренняя реализация выглядит примерно так:

```
1 class String {
2 private:
3     char* arr;
4     size_t sz;
5     size_t cap;
6 public:
7     String(size_t count, char c)
8     // Etc...
9 }
```

Конструктор копирования и оператор присваивания, условия их генерации компилятором. Для своих классов можно создавать:

- Copy constructor: Позволяет создать объект от другого объекта того же типа
- Assignment operator: Позволяет присвоить нам объект того же типа

Компилятор наряду с конструктором по умолчанию, также умеет генерировать за нас копирование объекта и операцию присваивания, и этот конструктор/оператор будет тривиальным. Те он поэлементно будет копировать все поля и все. Рассмотрим пример, который важен для понимания Copy constructor:

```
1 void f() {
2     String s(5, 'a');
3     String s2 = s; // (1)
4     String s3;
5     s3 = s; // (2)
6 }
```

Если вызвать в коде `f()`, то код упадет с ошибкой `double free`, так как тривиальные `copy-конструктор` (1) и `copy-assignment оператор` (2) просто скопируют указатель на данные `s` в поля `s2` и `s3`, а в конце скоупа каждый из них в своём деструкторе вызовет `delete[]` этих данных, таким образом деструкторы `s` и `s2` будут пытаться освободить память, которую уже освободил деструктор `s3`. Реализацию операторов можно посмотреть в примере для `String` ниже. Важное замечание оператор присваивания должен уметь корректно обрабатывать присваивание себе же; `s = f(s)`; если `f(s)` возвращает `s`

Поверхностное и глубокое копирование. При поверхностном копировании объект создается путем простого копирования всех полей исходного объекта. Это хорошо работает, если ни одна из переменных объекта не определена в разделе памяти `heap`. Если некоторым переменным динамически выделяется память из раздела кучи, то скопированная объектная переменная также будет ссылаться на ту же ячейку памяти. Это приведет к неоднозначности и ошибкам во время выполнения, а также к `dangling pointer`. Поскольку оба объекта будут ссылаться на одну и ту же ячейку

памяти, то изменения, внесенные одним из них, будут отражать эти изменения и в другом объекте. Поскольку мы хотели создать точную копию объекта, для этой цели не будет использоваться поверхностная копия. Примечание: если конструктор копирования/copy-assignment оператор не определены явно, компилятор C++ неявно их создаёт тривиальными, то есть выполняющими поверхностное копирование

Реализация копи-конструктора и оператора присваивания на примере класса String

Применяя идиому copy & swap

```

1 String(const String& other):
2     size_(other.size_), capacity_(other.capacity_), str_(new char[capacity_]) {
3     memcpy(str_, other.str_, size_);
4 }
5
6 String& operator=(String other) {
7     swap(other);
8     return *this;
9 }
10 ...
11 void swap(String& other) {
12     std::swap(size_, other.size_);
13     std::swap(capacity_, other.capacity_);
14     std::swap(str_, other.str_);
15 }
```

Возвращаемый тип оператора присваивания должен быть String&, чтобы можно было делать присваивание в цепочку s1 = s2 = s3;

Формулировка “правила трех”. Rule of Three. Это общеизвестное правило хорошего кода, которое гласит:

Если для класса определен хотя бы один из:

1. Нетривиальный конструктор копирования
2. Нетривиальный оператор присваивания
3. Нетривиальный деструктор

Значит, должны быть определены и все остальные

Условия генерации операторов компилятором. Конструктор по умолчанию: те же правила, что и в C++98. Генерируется только если класс не содержит определенных пользователем конструкторов.

Копирующий конструктор: то же поведение, что и в C++98: вызов копирующего конструктора для всех нестатических членов. Генерируется только если в классе нет определенного пользователем копирующего конструктора. Удаляется, если класс определяет операцию перемещения. Генерация этой функции в классе с определенным пользователем копирующим оператором присваивания или деструктором считается устаревшей (deprecated).

Копирующее присваивание: то же поведение, что и в C++98: копирующее присваивание всех нестатических членов. Генерируется только если в классе нет определенного пользователем копирующего присваивания. Удаляется, если в классе присутствует определенная пользователем операция перемещения. Генерация в случае, если в классе есть определенный пользователем копирующий конструктор или деструктор, считается устаревшей (deprecated).

17 Константные и статические методы

Понятие константного метода. Перегрузка между константными и неконстантными методами.

Определение оператора [] для String. Ключевое слово mutable для полей и его применение.

Понятие статических полей и методов, пример.

Константные методы Мещерин предлагает мыслить о константном объекте, следующим образом: константный тип такой же как и обычный, но у него убраны некоторые операции. Теперь мы хотим узнать как работать с константностью для наших собственных типов. Рассмотрим код:

```
1 struct S {
2     int x;
3     void f() {
4         std::cout << x;
5     }
6 };
7
8 int main {
9     const S s;
10    s.f()
11 }
```

В таком случае будет ошибка с пометкой "discards qualifiers" (Это означает что где-то в коде беда с константностью) Методы класса могут быть применимы к константным объектам или нет. В C++ если мы хотим чтобы метод присутствовал у константного объекта необходимо явно об этом написать, тк все методы по умолчанию считаются модифицирующими объект, пока не сказано обратное:

```
1 void f() const {}
```

Такой синтаксис сложился из-за исторических причин.

Если метод const, то, как правило, подразумевается что мы не меняем объект, те мы не можем делать неконстантные операции над полями, те поля считаются сами константами. Пример ошибки компиляции из-за нарушения данного правила (СЕ будет даже если не вызывать эту функцию):

```
1 void f() const {
2     ++x; // CE
3     std::cout << x;
4 }
```

Перегрузка между константными и неконстантными методами Если есть две функции константная и неконстантная, то будет обычная перегрузка.

Но все интереснее если поле класса это указатель

```
1 struct S {
2     int* p = new int(0);
3
4     void f() const {
5         *p = 2;
6     }
7 };
```

Такой код не вызовет ошибку. Так как константным считается сам указатель, а не то что под ним (те указатель выглядит как int* const p) текст

Также необходимо разбираться в случае ссылки

```

1 int x = 0; // Global variable
2
3 struct S {
4     int& a = x;
5
6     void f() const {
7         a = 1;
8     }
9 };

```

Как ни странно данный код скомпилируется. Тут нарушается наша интуиция по поводу ссылок. "а это другое имя для глобального икса. Константность навешивается как для указателей к самой ссылке, а не к тому что под ней(Потому что в реальности ссылка битово хранится как указатель)

Определение оператора [] для String. Для задачи String необходимо было переопределить оператор квадратных скобочек. И этот оператор должен себя по-разному вести себя для константных строк и не константных.

Для неконстантных строк этот оператор должен возвращать `char&`

А для константных должен возвращать `const char&`

Потому что результату оператора константных строк можно присваивать, а неконстантных нельзя.

```

1 char& operator[](size_t ind) {
2     return str_[ind];
3 }
4
5 const char& operator[](size_t ind) const {
6     return str_[ind];
7 }

```

Ключевое слово mutable для полей и его применение. Иногда бывает нужно, чтобы не смотря на константность метода, какое-то поле все равно было не константным(Аналогия friend function). Для этого есть специальное слово

```

1 struct S {
2     mutable int y = 0;
3
4     void f() const {
5         y = 1;
6     }
7 };

```

Данный код компилируется. Это применяется например в дебаге или в кэшировании

Понятие статических полей и методов, пример У полей есть специальное слово `static`:

```

1 struct S {
2     static int x; // Declaraion, not difinitioin
3 };

```

Для чего это нужно? Во-первых, он лежит в статической памяти. Во-вторых, это поле общее для всех объектов класса(Оно не у каждого отдельного объекта, а общее на весь класс).

(!) К НЕконстантным статическим полям есть одно странное требование. Оно должно быть инициализировано вне класса:

```

1 int S::x = 0; // Difinition

```

Это сделано из-за некоторых причин связанных с особенностями линковки (Чтобы избежать multiple definition при большом количестве инклюдов)

Для методов слово `static` уже ничего не говорит про расположение в памяти. Это просто по аналогии с полями, этот метод общий для всех объектов данного класса. Удобство в том, что для его пользования не нужно даже сосоздавать объект класса

```
1 struct S {  
2     static void f() {  
3         std::cout << "Heello!"  
4     }  
5 };  
6  
7 int main {  
8     S::f();  
9 }
```

Замечание, `const` для статических методов это СЕ тк это бессмыслица тк у статического объекта нет неявным параметром объекта, метод существует "Сам по себе"

"Канонический пример"это паттерн Singleton

```
3 // 3.7. Static fields and methods.  
4  
5 class Singleton {  
6 private:  
7     Singleton() {}  
8     Singleton(const Singleton&) = delete;  
9     static Singleton* ptr;  
10  
11 public:  
12     static Singleton& getInstance() {  
13         if (ptr == nullptr)  
14             ptr = new Singleton();  
15         return *ptr;  
16     }  
17 };  
18  
19 Singleton* Singleton::ptr = nullptr;  
20  
21 int main() {  
22     Singleton& s = Singleton::getInstance();  
23 }
```

18 Наследование

Идея наследования. Синтаксис определения наследования.

Модификатор доступа `protected` для членов класса и его смысл.

Особенности доступа к полям и методам класса-родителя и класса-наследника при публичном наследовании.

Размещение объектов в памяти и порядок вызова конструкторов при наследовании.

Порядок вызова конструкторов и деструкторов при наследовании.

Размещение в памяти объектов при наследовании.

Идея наследования. Синтаксис определения наследования. Некоторые типы являются частными случаями других типов. Если тип `Son`, наследник типа `Parent`, то мы ожидаем, что `Son` имеет все поля, может все то же, что и `Parent`, и возможно что-то еще.

```

1 struct Base{
2     int x;
3     void f() { std::cout << 1; }
4 }
5
6 struct Derived: Base { // ":" syntax for inheritance
7     int y;
8     void g() { std::cout << 2; }
9 }
```

Если мы создадим объект класса `Derived`, у него будут оба поля `x` и `y`, а также оба метода `f()` и `g()`. Тут есть некоторая аналогия с константностью. Наследник расширяет возможности по отношению к базовому классу.

Модификатор доступа `protected` для членов класса и его смысл. Вспомним какие у нас были до этого модификаторы доступа в классах/структурах:

- `private`. Это значит не доступно никому, кроме моих методов и моих друзей.
- `public`. Это доступно всем
- `protected`. Доступно моим методам и моим друзьям, а также моим наследникам. (`protected` = `private` + доступ наследникам)

Пример работы с `protected`:

```

1 struct Base{
2     int x;
3     protected:
4     void f(int x) { std::cout << x; }
5 };
6
7 struct Derived: Base {
8     int y;
9     void g(int a) { f(a + 1); } // OK
10 };
11
12 int main() {
13     Derived d;
14     d.f(1); // CE
15 }
```

То есть `f(tmp)` можно вызвать внутри наследованного класса, но нельзя извне.

Размещение объектов в памяти Когда мы наследуемся, мы говорим, что внутри нас есть полноценная часть Base. Там лежат все поля и методы. И только над этим мы делаем какую-то надстройку.

В памяти это также и хранится $[[x], y]$. Те сначала лежат все поля Base, потом поля Derived. Важно, что наследуются все поля, даже приватные, но просто доступа к ним нет.

Особенности доступа к полям и методам класса-родителя и класса-наследника при публичном наследовании.

Порядок вызова конструкторов и деструкторов при наследовании Когда создается объект наследника, сначала создаются все родители. Нельзя создать объект наследника, не создав при этом объектов родителей. Это делается перед тем, как будут инициализированы поля наследника. То есть

1. Создаются все поля родителя
2. Вызывается конструктор родителя (Если есть). Создания родителя завершено
3. Начинают создаваться поля наследника, вызывается конструктор наследника

Уничтожение идет в обратном порядке, сначала уничтожаются поля, потом деструктор наследника, далее уничтожается родитель.

```

5 struct A {
6     A(int x) { std::cout << "A created" << x << '\n'; }
7     ~A() { std::cout << "A destroyed" << '\n'; }
8 };
9
10
11 struct Base {
12     A a = 1;
13     Base() {
14         std::cout << "Base created" << '\n';
15     }
16     ~Base() {
17         std::cout << "Base destroyed" << '\n';
18     }
19 };
20
21 struct Derived: Base {
22     A a = 2;
23     Derived(int x) {
24         std::cout << "Derived created" << x << '\n';
25     }
26     ~Derived() {
27         std::cout << "Derived destroyed" << '\n';
28     }
29 };
30
31
32 int main() {
33     Derived d(5);
34 }

```

```

A created1
Base created
A created2
Derived created5
Derived destroyed
A destroyed
Base destroyed
A destroyed

```

Еще один момент в том, что если сделать у Base кастомный конструктор

```

1 struct Base{
2     A a;
3     Base(int x): a(x) {...}
4     ...
5 };

```

Будет ошибка компиляции, по причине того, что не понятно от чего создать Base. У нас нет дефолтного конструктора у Base, но создавая Derived, у нас нет инструкции компилятору, как создать Base по умолчанию, прежде чем создать Derived. Проблема решается тем, что мы должны руками прописать чем инициализировать Base:

```

1 struct Derived{
2     A a;
3     Derived(int x): Base(x) {...}
4     ...
5 };

```

Мы можем инициализировать только самого родителя, но не поля родителя, также нельзя прародителя

Возможно Мещерин опечатался в билетах имел в виду и наследование конструкторов, если поля Derived тривиально достраиваются. Для этого есть специальный синтаксис

```

1 struct Derived{
2     A a;
3     using Base::Base;
4     ...
5 };

```

Теперь у Derived есть все конструкторы Base. (!) Конструктор копирования не наследуется

19 Приведения типов при наследовании(в случае публично-го наследования)

Приведение типов между родителем и наследником: срезка при копировании, приведение указателей, приведение ссылок

Особенности приведения типов “вверх” и “вниз”. Как работает `static_cast` при наследовании?

Приведение типов между родителем и наследником: приведение указателей, приведение ссылок Мы хотим, чтобы был разрешен неявный каст, от наследника к родителю. Это буквально то ради чего мы и хотели делать наследование. Чтобы везде где ожидается `Base`, мы могли подложить частный случай `Base(Derived)`

```
1 struct Base {};
2 struct Derived: Base {};
3 void process(Base& b) {}
4
5 int main() {
6     Derived d;
7     Base& b = d; // (1)
8     process(d); // (2)
9 }
```

Рассмотрим (1) мы не создали новый объект, `b` — ссылка на ту часть `d`, в которой лежит отнаследованная часть `Base`. Демонстрация этой идеи, но уже при передаче в функции (2).

Неявный каст от ссылки/указателя на родителя к ссылке/указателю на сына запрещен. С помощью `static_cast` такой каст возможен, но если ссылка/указатель в самом деле указывают на объект `Base`, а не на объект `Derived`, то этот каст — UB

```
1 Derived& dd = b; // CE
```

Еще раз идея в том, что любой `Derived` является `Base`, но далеко не каждый `Base` является `Derived`. Тут есть аналогия с константными ссылками.

Данная философия работает и с указателями

```
1 Base* b = &d; // OK
2 Derived* dd = &b; // CE
```

Срезка при копировании В случае публичного наследования компилятор генерирует неявный конструктор `Base` от `Derived`

```
1 struct Base{
2     ...
3     Base(const Base&) {std::cout << Hi;}
4 };
5 int main() {
6     Derived d
7     Base b = d; // Slicing
8 }
```

Часть соответствующая `Base` просто копируется, с оставшейся частью `Derived` ничего не делается. Заметим, что в случае нетривиального конструктора копирования, он будет использован(Выведется `Hi`)

Особенности приведения типов “вверх” и “вниз”. Как работает `static_cast` Если нам дали `Base&`, но мы считаем что это должен быть `Derived`. То можно сделать явный каст (неявный запрещен). Для этого достаточно `static_cast`

```
1 int main() {  
2     Derived d  
3     Base& b = d;  
4     Derived& d2 = static_cast<Derived&>(b);  
5 }
```

Это корректно, после этого можно обращаться к полям и методам `d2`, как к полноценному объекту `Derived`. (Это корректно тк под `b` реально лежал `Derived`). Но если `Base b = d;` то это UB, тк `static_cast` не умеет проверять в Run Time. Аналогично можно кастить указатели на объекты. Все так работает только при публичном наследовании. Если наследование приватное, то никакой из вышеперечисленных кастов не работает, ни вверх, ни вниз. Только если `main` не друг `Derived`. Замечание: `private` наследование подразумевает, что из внешних функций мы не имеем права использовать тот факт, что `Derived` наследник `Base`. Все это будет СЕ по причине нарушения прав доступа.

Но `reinterpret_cast` может кастить в любом случае, ему вообще без разницы связаны ли типы хоть как-то

Не всегда в жизни А частный случай В, стоит делать через наследование [Circle-Ellipse problem](#).

Виды наследования Само наследование бывает также 3х типов(`public`, `protected`, `private`). Но в билете написано только про публичное наследование. Это 13 билет на хор

20 Шаблоны

Идея шаблонов и простые примеры.

Синтаксис определения шаблонов.

Шаблоны функций, пример. Шаблоны классов, пример.

Шаблонные объявления using. Шаблонные переменные. Шаблонные методы классов.

Синтаксис определения шаблонного метода шаблонного класса вне этого класса.

Синтаксис использования шаблонов.

Шаблоны — конструкция языка позволяющая использовать тип как переменный. Шаблоны по своей сути являются инструкциями для компилятора, как именно нужно сгенерировать функции по данному типу. В виду этого раскрытие шаблонов происходит в compile time.

Шаблон функции:

```
1 template <typename T>
2 T max(T x, T y) {
3     return x > y ? x : y;
4 }
```

Шаблон класса:

```
1 template <typename T>
2 class vector {
3     T* arr;
4     size_t sz;
5     size_t cap;
6 }
```

Шаблонный using:

```
1 template <typename T>
2 using mymap = std::map<T, T, std::greater<T>>;
```

Шаблонные переменные:

```
1 template<class T>
2 constexpr T pi = T(3.1415);
```

Шаблонные методы класса можно объявлять внутри, а можно объявлять вне класса, но в таком случае нужно заново объявить шаблонные параметры.

```
1 template <typename T>
2 class A {
3 public:
4     template <typename U>
5     void kek(T a, U b) {
6         std::cout << "kek\n";
7     }
8
9     template <typename U>
10    void lol(T a, U b);
11 };
12
13 template <typename T>
14 template <typename U>
15 void A<T>::lol(T a, U b) {
16     std::cout << "lol\n";
17 }
```

Синтаксис использования шаблонов:

Можно заметить что в шаблонных функциях шаблонный аргумент может быть выведен самостоятельно, но преобразований при этом не совершается.

```

1 int main() {
2     int i = max(5, 0);
3     long long l = max<long long>(111, 011);
4     vector<std::string> v;
5     mymap<size_t> m;
6     array<char, 5> arr;
7     A<int> a;
8     a.kek<int>(1, 2);
9     a.lo1<int>(1, 2);
10 }
```

21 Шаблоны с числовыми параметрами

Требования к числам в параметрах шаблона.

Пример: класс array.

Еще пример: класс матриц, объявление оператора умножения матриц.

До C++20 числовые параметры в шаблонах могли быть только целочисленными типами, начиная же с C++20 они также могут быть типами с плавающей точкой.

Класс array:

```

1 template <typename T, size_t Size>
2 class array {
3     T arr[Size];
4 }
```

Класс матриц:

```

1 template <typename T, size_t Height, size_t Width>
2 class matrix {
3     T mat[Height][Width];
4 };
5
6 template <typename T, size_t N, size_t M, size_t K>
7 matrix<T, N, K> operator*(matrix<T, N, M> first, matrix<T, M, K> second) {
8     // some code here
9 }
```

22 Полиморфизм и виртуальные функции

Понятие полиморфного типа. Синтаксис объявления виртуальных функций. Что такое виртуальная функция? Чем виртуальные функции отличаются от обычных. Зачем нужно слово `override`? Что, если сигнатуры функции у родителя и у наследника слегка отличаются (например, словом `const`)? Ключевое слово `final` и его применение.

Тип называется *полиморфным*, если у него существует хотя бы один виртуальный метод или хотя бы один унаследованный виртуальный метод.

```
1 class A {
2     virtual void f() {
3         // some code here
4     }
5
6     virtual ~A() = default;
7 }
```

Добавление спецификатора `virtual` в начало объявления метода делает его *виртуальным*, что означает, что его вызов у объекта по ссылке на родителя все равно приведет к работе версии наследника.

Спецификатор *`override`* говорит о том, что данный метод является переопределением некоторого метода из родителя, что заставляет компилятор явно проверить это и выдать ошибку, если это не так.

При переопределении сигнатура переопределяемого метода должна полностью совпадать, иначе это будет считаться другим методом.

Здесь необходимо уточнить деталь, что равенство сигнатур определяется с точностью до навешивания `const` на верхний уровень аргументов. Ну или нет, так как это решение было найдено в ходе обсуждения контрпримера к изначальной правоте:

```
1 class A {
2 public:
3     virtual void f(int a) {
4         std::cout << 1;
5     }
6     virtual ~A() = default;
7 };
8
9 class B : public A {
10 public:
11     void f(const int a) override {
12         std::cout << 2;
13     }
14     virtual ~B() = default;
15 };
16
17 int main() {
18     B b;
19     A& a = b;
20     int t = 5;
21     a.f(t); // prints 2
22 }
```

Спецификатор *`final`* говорит что метод является последним переопределением, то есть что у дочерних классов данный метод переопределяться не будет. В случае если это не так компилятор выдаст ошибку.

23 Абстрактные классы и чисто виртуальные функции

Понятие `pure virtual` функций, синтаксис их объявления.

Понятие абстрактного класса и его особенности. Зачем нужен виртуальный деструктор? Какие могут быть плохие последствия, если его забыть?

Виртуальные функции и приватность: что происходит, если приватность родительской и дочерней версии различается?

Виртуальная функция к которой справа приписано `= 0` называется *pure virtual* функцией.

```
1 class A {  
2     virtual void f() = 0;  
3 };
```

Класс имеющий хотя бы одну `pure virtual` функцию называется *абстрактным*. Это подразумевает, что некоторые его методы будут переопределять дочерние классы, и при этом сам класс не будет иметь их реализации. Также это подразумевает что объект данного типа нельзя создать. Наследник, в случае если он не переопределяет `pure virtual` функции остается абстрактным.

Рассмотрим следующий пример:

```
1 class Base {};  
2  
3 class Derived : public Base {  
4     int* p = new int(0);  
5     ~Derived() = default;  
6 };  
7  
8 int main() {  
9     Base* b = new Derived();  
10    delete b;  
11 }
```

В этом коде происходит утечка памяти, ведь деструктор вызовется у объекта с точки зрения `Base`, а значит выделенная память не очистится. Для того, чтобы решить эту, проблему используют виртуальные деструкторы. Правилом хорошего тона будет всегда писать виртуальный деструктор у полиморфного класса.

```
1 class Base {  
2 public:  
3     virtual ~Base() = default;  
4 };  
5  
6 class Derived : public Base {  
7 public:  
8     int* p = new int(0);  
9     virtual ~Derived() = default;  
10 };  
11  
12 int main() {  
13     Base* b = new Derived();  
14     delete b;  
15 }
```


Важно помнить, что проверка приватности — это этап, который выполняется в compile time, когда выбор версии функции среди переопределений — в runtime. В виду этого если выражение корректно с точки зрения компиляции, то может вызываться даже приватный метод класса.

```
1 class Base {
2 public:
3     virtual void f() {
4         std::cout << 1;
5     }
6     virtual ~Base() = default;
7 };
8
9 class Derived : public Base {
10 private:
11     void f() {
12         std::cout << 2;
13     }
14 public:
15     virtual ~Derived() = default;
16 };
17
18 int main() {
19     Derived d();
20     Base& b = d;
21     b.f(); // prints 2
22 }
```

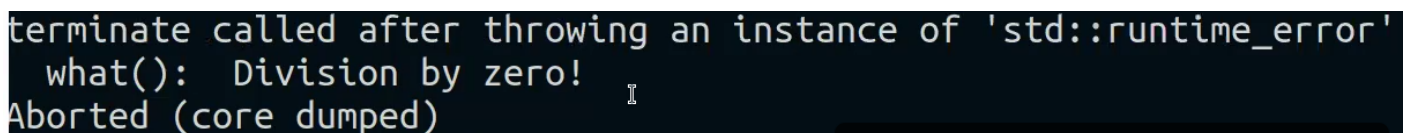
24 Исключения

Мотивировка использования исключений. Оператор `throw` и конструкция `try...catch`. Понятие `stack unwinding` (“сворачивание стека”), функция `std::terminate`. Примеры стандартных операторов и функций, генерирующих исключения. Разница между RE и исключениями. Примеры RE, не являющихся исключениями. Стандартные классы для исключений, тип `std::runtime_error` и другие.

Мотивировка использования исключений. Предположим, у нас есть функция, совершающая некоторое полезное действие. Как понять, успешно ли она его совершила/частично успешно/-полностью неуспешно? В языке C проблема решалась следующим образом: добавлением специфического кода возврата функции и/или складыванием результата выполнения по какому-нибудь указателю. Но это не очень удобно, поскольку в таком случае каждый вызов функции, в которой может пойти что-то не так надо оборачивать `if`-ами на всевозможные коды возврата. (Да и если `int f(int)`; — код возврата может быть и обычным числом, например `atoi`). Для решения этой проблемы была придумана концепция исключений (“exception”): если функция может потенциально завершиться неудачно, то вместо возврата специального кода, она может нештатно завершиться и пробросить вверх ошибку.

Оператор `throw` и конструкция `try...catch`. Понятие `stack unwinding` (“сворачивание стека”), функция `std::terminate`. Для этого используется оператор `throw` (по сути, он является альтернативным `return`’у способом выйти из функции), который и пробрасывает исключение вверх, например:

```
1 int divide(int a, int b) {
2     if (b == 0) {
3         throw std::runtime_error("Division by zero");
4     }
5     return a / b;
6 }
```



```
terminate called after throwing an instance of 'std::runtime_error'
what(): Division by zero!
Aborted (core dumped)
```

При выходе из функции посредством исключения вызывается функция `std::terminate`: её задача — это сообщить нам о том, что что-то пошло не так (и показать что именно пошло не так: т.е. выводится класс исключения и строка, переданная исключению) и аварийно завершить программу вызовом `std::abort` (C-ая функция, сообщает системе о том, что лавочку пора закрывать). Но этого вызова можно избежать, если обернуть потенциально опасное место в `try...catch`:

```
1 int main() {
2     int x;
3     int y;
4     std::cin >> x >> y;
5     try {
6         std::cout << divide(x, y);
7     } catch (std::runtime_error& err) {
8         std::cout << "caught! " << err.what();
9     }
10 }
```

И тогда при вводе invalidных аргументов мы поймем исключение: Так же, ловить мы можем не только какой-то конкретный тип исключений, но и просто все:

```
5
9
caught! Division by zero!
```

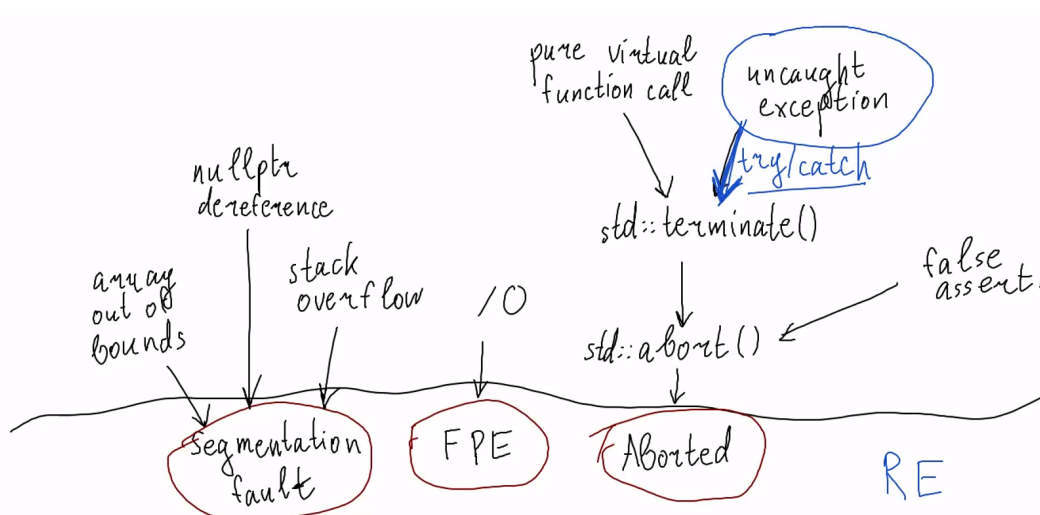
```
1 int main() {
2     int x;
3     int y;
4     std::cin >> x >> y;
5     try {
6         std::cout << divide(x, y);
7     } catch (...) {
8         std::cout << "caught! ";
9     }
10 }
```

При выходе из функции посредством throw у нас уничтожаются все локальные объекты на текущем уровне вложенности, а потом мы поднимаемся на уровень и выше, и, если там нас не поймали, проделываем аналогичную операцию, et cetera, и так до тех пор, пока не вылетим из main и сработает std::terminate или exception не будет пойман каким-то catch.

Разница между RE и исключениями. Примеры RE, не являющихся исключениями. Не всё, что является RE – исключение. Например : segfault, FPE, aborted. Они не ловятся при помощи try...catch, поскольку сами по себе не являются исключениями.

- К segfault может привести array out of bounds, разыменование nullptr, stack overflow.
- К FPE приводит деление на 0.
- К aborted приводит вызов функции std::abort. А вот к вызову функции std::abort приводит, например, вызов функции std::terminate, false assertion.

И вот одним из случаев, в котором вызывается std::terminate является uncaught exception (Или может вызваться из-за, for example, pure virtual function call). Из этого легко увидеть, что исключения – это не более чем частный случай RE.



Стандартные классы для исключений, тип std::runtime_error и другие. Примеры стандартных операторов и функций, генерирующих исключения. В стандартной библиотеке C++ есть базовый класс std::exception, от которого унаследованы все стандартные исключения. Есть несколько больших классов исключений, например – logic_error. Условно, стоит воспринимать этот класс как ошибку на стороне клиента, т.е. пользователь сделал что-то не то.

- `logic_error`
 - `invalid_argument`
 - `domain_error`
 - `length_error`
 - `out_of_range`
 - `future_error` (since C++11)

Исключение `out_of_range` кидается, например функцией `at` в `std::vector` (да и просто в стандартных контейнерах с индексацией).

Не стоит обманываться названием класс `runtime_error`: его можно поймать и никакого RE не будет. Этот класс exception можно воспринимать как исключение со стороны сервера: случилось что-то не так в коде.

- `runtime_error`
 - `range_error`
 - `overflow_error`
 - `underflow_error`
 - `regex_error` (since C++11)
 - `system_error` (since C++11)
 - `ios_base::failure` (since C++11)
 - `filesystem::filesystem_error` (since C++17)
 - `tx_exception` (TM TS)
 - `nonexistent_local_time` (since C++20)
 - `ambiguous_local_time` (since C++20)
 - `format_error` (since C++20)

Так же существуют исключения, не лежащие ни в одном из этих двух больших классов. Они, по большей части, вызываются какими-то стандартными операторами: Например, `dynamic_cast`

- `bad_typeid`
- `bad_cast`
 - `bad_any_cast` (since C++17)
- `bad_optional_access` (since C++17)
- `bad_expected_access` (since C++23)
- `bad_weak_ptr` (since C++11)
- `bad_function_call` (since C++11)
- `bad_alloc`
 - `bad_array_new_length` (since C++11)
- `bad_exception`
- `ios_base::failure` (until C++11)
- `bad_variant_access` (since C++17)

бросает `bad_cast` если каст к ссылке неудачен (если неудачен каст к указателю, то возвращается `nullptr`, но ссылке необходимо чем-то проинициализировать); `bad_type_id` бросается при вызове `typeid` от `nullptr`; `bad_alloc` – бросается `new`, если он не смог выделить память.

Примечание: примерчиков маловато? И не маловато на удос?

25 Идиома RAII

Проблема утечки памяти при исключениях, мотивировка RAII.

Умные указатели как реализация RAII.

Базовое использование классов `std::unique_ptr` и `std::shared_ptr`:

основные методы и их применение (без реализации).

Можно ли использовать `unique_ptr` в контейнерах?

Проблема утечки памяти при исключениях, мотивировка RAII. Рассмотрим следующий код, который позволяет осознать некоторую проблему концепции exception'ов:

```

1 void g(int x) {
2     if (x == 0) {
3         throw 1;
4     }
5 }
6
7 void f(int x) {
8     int* p = new int(x);
9     g(*p); // throws
10    delete p;
11 }
12
13 int main() {
14     int x;
15     std::cin >> x;
16     f(x);
17 }
```

Что мы здесь видим? У нас выделяется в динамической памяти `int`, а потом может быть брошено исключение. Но тут возникает проблема: выделенный `int` никуда не денется, а значит возникает memory leak. А значит возникает серьёзная (собственно, главная) проблема исключений: теперь мы не можем быть уверены, что выход из функции произойдёт в должном месте, а захваченный ресурс будет корректно освобождён.

Одним из способов решения этой проблемы является отказ от исключений в принципе. Но, есть и другой способ решения: RAII.

RAII – Resource Acquisition Is Initialisation (захват ресурса – его инициализация);

Реализация этой идиомы состоит в оборачивании захваченных ресурсов в некоторую обёртку, которая позволит их удалять при ловле exception'ов:

```

1 struct Wrapper {
2     int* p;
3     Wrapper(int* p): p(p) {}
4     ~Wrapper() { delete p; }
5 }
```

А функция `f` модифицируется следующим образом:

```

1 void f(int x) {
2     Wrapper p(new int(x));
3     g(*p);
4 }
```

И теперь у нас не будет утечки памяти.

В итоге, получается, что идиома RAII заключается в:

- обёртывании каждого ресурса в класс, где
 - конструктор получает права на ресурс, и инициализирует все поля класса или кидает exception в случае невозможности успешного завершения создания объекта

- деструктор корректно освобождает ресурс и никогда не кидает exception's.
- использовании каждого ресурса только через инстанс RAII-класса, который либо
 - сам имеет ограниченное время жизни/заранее заданное время хранения, либо
 - имеет время жизни, связанное либо с некоторым автоматическим/временным объектом

Умные указатели как реализация RAII. Класс Wrapper является примитивным умным указателем, т.е. указателя-обёртки над настоящим поинтером, основной задачей которого является корректное уничтожение настоящего pointer'a в момент уничтожения себя – т.е. это RAII класс для указателей на объекты.

Базовое использование класса `std::unique_ptr`: основные методы и их применение (без реализации). Одним из примеров реализации концепции умных указателей в STL служит `unique_ptr`:

```

1 template<typename T>
2 struct unique_ptr {
3     T* ptr = nullptr;
4     unique_ptr() {}
5     unique_ptr(const unique_ptr&)=delete;
6     unique_ptr& operator=(const unique_ptr&)=delete;
7     unique_ptr(T* p): ptr(p) {}
8     ~unique_ptr() { delete ptr; }
9     T* get() { return ptr; }
10    void reset() {
11        delete ptr;
12        ptr = nullptr;
13    }
14 }
```

(Выше приведён примитивный вариант реализации `unique_ptr`) Чем вообще характеризуется `unique_ptr`? Указателем на один ресурс владеет только один `unique_ptr`, и его нельзя копировать. Мувать можно. Как следствие, его нельзя принимать по значению.

Member functions	
(constructor)	constructs a new <code>unique_ptr</code> (public member function)
(destructor)	destructs the managed object if such is present (public member function)
operator=	assigns the <code>unique_ptr</code> (public member function)
Modifiers	
release	returns a pointer to the managed object and releases the ownership (public member function)
reset	replaces the managed object (public member function)
swap	swaps the managed objects (public member function)
Observers	
get	returns a pointer to the managed object (public member function)
get_deleter	returns the deleter that is used for destruction of the managed object (public member function)
operator bool	checks if there is an associated managed object (public member function)
Single-object version, <code>unique_ptr<T></code>	
operator* operator->	dereferences pointer to the managed object (public member function)
Array version, <code>unique_ptr<T[]></code>	
operator[]	provides indexed access to the managed array (public member function)

Рис. 1: Основные методы `unique_ptr`

Приведём примеры использования `unique_ptr`:

```
std::unique_ptr<T, Deleter>::operator=
```

unique_ptr& operator=(unique_ptr&& r) noexcept;	(1)	(constexpr since C++23)
template< class U, class E > unique_ptr& operator=(unique_ptr<U, E>&& r) noexcept;	(2)	(constexpr since C++23)
unique_ptr& operator=(std::nullptr_t) noexcept;	(3)	(constexpr since C++23)
unique_ptr& operator=(const unique_ptr&) = delete;	(4)	

Рис. 2: Конструкторы unique_ptr

```
std::unique_ptr<T, Deleter>::unique_ptr
```

members of the primary template, unique_ptr<T>		
constexpr unique_ptr() noexcept;	(1)	
constexpr unique_ptr(std::nullptr_t) noexcept;		
explicit unique_ptr(pointer p) noexcept;	(2)	(constexpr since C++23)
unique_ptr(pointer p, /* see below */ d1) noexcept;	(3)	(constexpr since C++23)
unique_ptr(pointer p, /* see below */ d2) noexcept;	(4)	(constexpr since C++23)
unique_ptr(unique_ptr&& u) noexcept;	(5)	(constexpr since C++23)
template< class U, class E > unique_ptr(unique_ptr<U, E>&& u) noexcept;	(6)	(constexpr since C++23)
unique_ptr(const unique_ptr&) = delete;	(7)	
template< class U > unique_ptr(std::auto_ptr<U>&& u) noexcept;	(8)	(removed in C++17)
members of the specialization for arrays, unique_ptr<T[]>		
constexpr unique_ptr() noexcept;	(1)	
constexpr unique_ptr(std::nullptr_t) noexcept;		
template< class U > explicit unique_ptr(U p) noexcept;	(2)	(constexpr since C++23)
template< class U > unique_ptr(U p, /* see below */ d1) noexcept;	(3)	(constexpr since C++23)
template< class U > unique_ptr(U p, /* see below */ d2) noexcept;	(4)	(constexpr since C++23)
unique_ptr(unique_ptr&& u) noexcept;	(5)	(constexpr since C++23)
template< class U, class E > unique_ptr(unique_ptr<U, E>&& u) noexcept;	(6)	(constexpr since C++23)
unique_ptr(const unique_ptr&) = delete;	(7)	

Рис. 3: Assignment операторы для unique_ptr

```
1 #include <iostream>
2 #include <memory>
3
4 int main() {
5     std::unique_ptr<int> ptr = std::make_unique<int>(42);
6     std::cout << *ptr << std::endl;
7 }

1 #include <memory>
2 template <typename T>
3 class ForwardList {
4 private:
5     struct Node {
6         T data;
7         std::unique_ptr<Node> next;
8     };
9     std::unique_ptr<Node> head;
10 public:
11     void PushFront(const T& elem) {
12         head = std::make_unique<Node>(elem, std::move(head));
13     }
14     void PopFront() {
15         head = std::move(head->next);
16     }
17     const T& Front() const {
18         return head->data;
19     }
20     bool Empty() const {
21         return head == nullptr;
22     }
23     ~ForwardList() {
24         while (!Empty()) {
25             PopFront();
26         }
27     }
28 }
```

```

27     }
28 };

```

Можно ли использовать `unique_ptr` в контейнерах? Скажем так, чисто технически использовать можно, но многие операции станут невозможными; после помещения `unique_ptr` его создание возможно, в случае, например, `std::vector` можно использовать и большинство его функций, но для этого необходимо будет постоянно мувать объект (кроме получения доступа), что, идеологически, наверное не очень хорошо.

```

1  typedef std::unique_ptr<int> unique_t;
2  typedef std::vector< unique_t > vector_t;
3
4  vector_t vec1;                                // fine
5  vector_t vec2(5, unique_t(new Foo));          // Error (Copy)
6  vector_t vec3(vec1.begin(), vec1.end());      // Error (Copy)
7  vector_t vec3(make_move_iterator(vec1.begin()), make_move_iterator(vec1.end()));
8
9  std::sort(vec1.begin(), vec1.end()); // fine, because using Move Assignment
   Operator
10
11 std::copy(vec1.begin(), vec1.end(), std::back_inserter(vec2)); // Error (copy)

```

Базовое использование класса `std::shared_ptr`: основные методы и их применение (без реализации). Шеред же реализует другую идею, идею разделяемого владения каким-то объектом с подсчётом указателей на него и удалением только с удалением всех указателей.

Member functions	
(constructor)	constructs new <code>shared_ptr</code> (public member function)
(destructor)	destructs the owned object if no more <code>shared_ptr</code> s link to it (public member function)
<code>operator=</code>	assigns the <code>shared_ptr</code> (public member function)
Modifiers	
<code>reset</code>	replaces the managed object (public member function)
<code>swap</code>	swaps the managed objects (public member function)
Observers	
<code>get</code>	returns the stored pointer (public member function)
<code>operator*</code> <code>operator-></code>	dereferences the stored pointer (public member function)
<code>operator[]</code> (C++17)	provides indexed access to the stored array (public member function)
<code>use_count</code>	returns the number of <code>shared_ptr</code> objects referring to the same managed object (public member function)
<code>unique</code> (until C++20)	checks whether the managed object is managed only by the current <code>shared_ptr</code> object (public member function)
<code>operator bool</code>	checks if the stored pointer is not null (public member function)
<code>owner_before</code>	provides owner-based ordering of shared pointers (public member function)
<code>owner_hash</code> (C++26)	provides owner-based hashing of shared pointers (public member function)
<code>owner_equal</code> (C++26)	provides owner-based equal comparison of shared pointers (public member function)

Рис. 4: Основные методы `shared_ptr`

```

1  #include <iostream>
2  #include <memory>
3
4  int main() {
5      std::shared_ptr<int> ptr1 = std::make_shared<int>(17);
6      std::cout << *ptr1 << "\n"; // 17
7      std::cout << ptr1.use_count() << "\n"; // 1
8
9      auto ptr2 = ptr1; // copy allowed!
10     std::cout << *ptr1 << "\n"; // 17
11     std::cout << *ptr2 << "\n"; // 17 - same object

```



```
12     std::cout << ptr1.use_count() << "\n";    // 2
13     std::cout << ptr2.use_count() << "\n";    // 2
14
15     std::shared_ptr<int> ptr3;
16     std::cout << ptr3.use_count() << "\n";    // 0
17
18     ptr3 = ptr1;    // assignment allowed!
19     std::cout << *ptr3 << "\n";    // 17
20     std::cout << ptr1.use_count() << "\n";    // 3
21     std::cout << ptr2.use_count() << "\n";    // 3
22     std::cout << ptr3.use_count() << "\n";    // 3
23 }
```

Примерчики не айс, понимаю, но что-то ничего нормального сходу не пришло в голову

26 Контейнеры

Последовательные и ассоциативные контейнеры.

Контейнеры `vector`, `deque`, `list`, `forward_list`, `map`, `set`, `unordered_map`, `unordered_set` (без реализаций).

Поддерживаемые операции и асимптотики операций в каждом из контейнеров.

Алгоритмы и структуры данных, используемые в каждом из контейнеров.

Адаптеры над контейнерами: `stack`, `queue`, `priority_queue`, их использование.

Последовательные и ассоциативные контейнеры Контейнеры STL делятся на три типа:

- Sequence container – последовательный контейнер, т.е. структура данных, доступ к элементам которой может быть получен последовательно (порядок элементов зависит от порядка добавления/удаления, не от значения)
 - `array`
 - `vector`
 - `deque`
 - `forward_list`
 - `list`
- Associative container – ассоциативные структуры данных, в которых легко выполняется поиск элемента ($O(\log n)$)
 - `set`
 - `map`
 - `multiset`
 - `multimap`
- Unordered associative container – неупорядоченные ассоциативные структуры данных, хеш-таблицы по сути, в которых поиск выполняется за $O(1)$ в среднем, но за $O(n)$ в худшем случае
 - `unordered_set`
 - `unordered_map`
 - `unordered_multiset`
 - `unordered_multimap`

`std::vector`: поддерживаемые операции, асимптотики, алгоритмы/структуры в контейнере Реализован на массиве постоянной длины, который реаллоцируется в случае необходимости расширения, имеет `ContiguousAccess` итератор; имеет специализацию `vector<bool>` – просто динамический `bitset`. Соответственно, можем получать доступ к элементу по индексу, добавлять/удалять в конец за учётную единицу; но, при перевыделении памяти у нас всё инвалидируется.

std::deque: поддерживаемые операции, асимптотики, алгоритмы/структуры в контейнере std::deque – последовательный индексируемый контейнер, главной фишкой которого является возможность добавления/удаления в начало/с конца в среднем за единицу; внутренним устройством является массив указателей на bucket'ы, в которых хранятся элементы; при нужде, мы реаллоцируем этот массив, перемещая имеющиеся указатели на bucket'ы в середину. Собственно, поскольку мы сами bucket'ы не трогаем, то при удалении/добавлении с краёв не инвалидируются ссылки на оставшиеся элементы. В то же время, опять-таки из внутреннего устройства следует, что для получения элемента требуется в среднем одно разыменование. Имеет RandomAccess итераторы

С точки зрения методов мало чем отличается от вектора, кроме возможности добавлять и удалять сначала и отсутствия data() (ну, у нас и категория итератора для data не Contiguous)

std::forward_list, std::list: поддерживаемые операции, асимптотики, алгоритмы/структуры в контейнере std::forward_list – односвязный список на нодах; соответственно, позволяет быстро добавлять элементы в любое место, но нет доступа по индексу; т.к. односвязный, то только ForwardIterator. Не инвалидируются итераторы и ссылки на неудалённые элементы. Есть специфические методы merge – объединение двух отсроченных, splice_after – перемещаем элементы из другого forward_list, reverse, unique – линия, sort – $O(n \log n)$.

std::list – двусвязный список на нодах с fake_node; можно быстро добавлять/удалять элементы в любом месте, имеет категорию итератора BidirectionalIterator; ссылки и итераторы на неудалённые элементы не инвалидируются. Специфические методы те же, что и у forward_list.

std::map, std::set: поддерживаемые операции, асимптотики, алгоритмы/структуры в контейнере std::set – КЧД с поддержкой кастомного компаратора; поиск, удаление, вставка – $O(\log n)$; есть возможность merge двух set'ов, есть lower_/upper_bound.

std::map – КЧД на парах со сравнением по уникальным ключам; собственно, операции аналогичны set'у, асимптотика та же.

Итерирование происходит в порядке увеличения ключей, категория итератора – Bidirectional

std::unordered_set, std::unordered_map: поддерживаемые операции, асимптотики, алгоритмы/структуры в контейнере std::unordered_set, std::unordered_map – хеш-мапы, в среднем добавляем/ищем/удаляем за единицу, в худшем случае – за линию; категория итератора – ForwardIterator.

Адаптеры над контейнерами: stack, queue, priority_queue

std::stack, std::queue, std::priority_queue – сами по себе не контейнеры, а только адаптеры над ними, дающие определённую функциональность используя методы внутреннего контейнера. Если мы переопределяем внутренний контейнер, то мы передаём typename, т.е. уже готовый тип.

- std::stack – LIFO (last in first out), дефолтным внутренним контейнером является дек, но можно использовать любой последовательный контейнер с функциями back(), pop_back(), push_back().
- std::queue – FIFO (first in first out), дефолтным внутренним деком, но можно использовать любой последовательный контейнер с функциями back(), front(), push_back(), push_front().
- std::priority_queue – heap, по умолчанию внутри дек, но можно использовать любой последовательный контейнер с RandomAccess категорией итераторов и функциями front(), push_back(), pop_back().

27 Итераторы, категории итераторов

Range-based for, его принцип работы и использование.

Какие категории итераторов существуют?

Какие категории итераторов в каждом из контейнеров?

Примеры использования итераторов в стандартных алгоритмах.

Примеры алгоритмов, требующих определенные виды итераторов.

Какие алгоритмы из `<algorithm>` применимы, а какие неприменимы к каждому из контейнеров (достаточно нескольких примеров).

Итераторы и их категории Итераторы – объекты, используя которые мы можем идентифицировать и обходить элементы контейнера. Собственно, поэтому итератором называется любой тип, который можно разыменовывать и инкрементировать. Но есть частные случаи итераторов, которые удовлетворяют более строгим требованиям:

- `InputIterator` – такой итератор, для которого определён оператор `==`. Он не гарантирует валидность более чем однопроходных алгоритмов.
- `OutputIterator` – такой итератор, который может выполнять запись в элемент, на который указывает. Аналогично `InputIterator`, валидность его использования в многопроходных алгоритмах не гарантируется
- `ForwardIterator` – такой `InputIterator`, который гарантирует, что при повторном проходе у нас ничего не сломается. Эту категорию имеют `forward_list`, `unordered_set`, `unordered_map`
- `BidirectionalIterator` – такой `ForwardIterator`, по которому мы можем шагать и в обратную сторону. Эту категорию имеет `std::list`, `std::set`, `std::map`.
- `RandomAccessIterator` – такой `BidirectionalIterator`, для которого определены операторы `+=`, `-=`, `++` с числом, `</>`/`<=`/`>=` с другим итератором, причём перемещение на произвольный элемент должно происходить за константное время. Эту категорию имеет `std::deque`
- `ContiguousIterator` – такой итератор, который по сути аналогичен указателю, т.е. $*(it + n) = *(& *it + n)$. Эту категорию имеют `std::vector`, `std::array`

Range-based for Начиная с C++11 вместо подобной конструкции

```
1 std::set<int> s;
2 for (std::set<int>::iterator it = s.begin(); it != s.end(); ++it) {
3     // some code...
4 }
```

Мы можем использовать

```
1 sometype s;
2 for (auto x : s) {
3     // some code...
4 }
```

Для типа, в котором определены функции `begin()`, `end()` возвращающие итераторы.

Также, начиная с C++17 итерируясь по, допустим, `unordered_map` мы можем писать вот так:

```
1 for (auto&& [key, value]: somemap) {
2     // some code...
3 }
```

Алгоритмы из `algorithm` и их применение к стандартным контейнерам, примеры использования итераторов в стандартных алгоритмах

- Стандартная STL'ая сортировка основана на quicksort и требует RandomAccess итераторы для работы;

```
1 std::sort(some_vector.begin(), some_vector.end())
2
```

В случае вызова от Bidirectional или более общего итератора наиболее вероятным будет UB/СЕ. Поэтому нельзя отсортировать `std::list`, `std::map`, etc, но можно отсортировать `deque` и `vector`.

- А, допустим, для копирования с использованием `std::copy` нам достаточно только InputIterator'а, поэтому мы можем применить `std::copy` для итераторов всех стандартных контейнеров (и не только)

```
1 int main() {
2     std::vector<int> from_vector(10);
3     std::iota(from_vector.begin(), from_vector.end(), 0);
4
5     std::vector<int> to_vector;
6     std::copy(from_vector.begin(), from_vector.end(),
7               std::back_inserter(to_vector));
8 }
9
```

- Поиск подпоследовательности в некоторой структуре при помощи итератора и `std::search` требует как минимум ForwardIterator, поскольку нам нужна гарантия неизменности элементов в контейнере.
- Для разворачивания некоторой последовательности элементов нам нужен BidirectionalIterator, поэтому функция `std::reverse` неприменима к `std::forward_list` и `std::unordered_map`, `std::unordered_set`.

```
1 int main() {
2     std::vector<int> v {1, 2, 3};
3     std::reverse(v.begin(), v.end());
4 }
5
```

- Функция `std::lower_bound` формально требует ForwardIterator, и применима ко всем стандартным контейнерам; но если данный на вход итератор не RandomAccess, то количество инкрементов итератора будет линейным (поэтому в случае наличия у контейнера встроенной функции `lower_bound` её использование предпочтительнее)

```
1 int main() {
2     const std::vector<int> data{1, 2, 4, 5, 5, 6};
3
4     for (int i = 0; i < 8; ++i)
5     {
6         // Search for first element x such that i ≤ x
7         auto lower = std::lower_bound(data.begin(), data.end(), i);
8         std::cout << i << " ≤ ";
9         lower != data.end()
10            ? std::cout << *lower << " at index " << std::distance(data.begin(), lower)
11            : std::cout << "not found";
12         std::cout << '\n';
13     }
14 }
```

- `std::next_permutation` требует для своего применения `BidirectionalIterator`, поэтому её вызов к `unordered_map` вызовет СЕ.

```
1 int main()
2 {
3     std::string s = "aba";
4
5     do
6     {
7         std::cout << s << '\n';
8     }
9     while (std::next_permutation(s.begin(), s.end()));
10
11     std::cout << s << '\n';
12 }
13
```

При выполнении этой программы выведется: aba, baa, aab.

Можно ещё примерчиков накидать...

28 Move-семантика

Проблемы, приводящие к идее move-семантики.
 Метод `emplace_back` и его отличие от `push_back`.
 Функция `std::move` и примеры, когда она уместна.
 Move-конструкторы и move-assignment операторы.
 Поддержка move-семантики для пользовательских классов.
 Реализация move-конструктора и move-assignment оператора на примере класса `String`.
 Условия их генерации компилятором и их действие в этом случае.
 “Правило пяти” вместо “правила трех”.

Начало move-семантики в лекциях продвы

Проблема, приводящая к move-семантике

Рассмотрим следующий код и зададимся вопросом, сколько объектов типа `std::string` в нём создается:

```
1 std::vector<std::string> v;
2 v.push_back("abc");
```

Метод `push_back` имеет сигнатуру `void push_back(const T& value)`, а внутри него в какой-то момент будет вызвано `new(arr + n) T(value)`

Таким образом, конструктор `std::string` будет вызван дважды: сначала конструктор временной строки из строкового литерала, а внутри `push_back` будет вызван конструктор копирования

Та же проблема распространяется на любое сохранение данных в любые пользовательские классы.

```
1 struct S {
2     std::string data;
3     S(const std::string& str) : data(str) {}
4 };
5 S s("abc");
```

Используя только знания, имеющиеся на текущий момент, нет способа проинициализировать данные в `S` напрямую из аргументов, не создавая временный объект.

В качестве ещё одного примера вспомним функцию `swap`. Наивная реализация `swap` может выглядеть так:

```
1 template <typename T>
2 void swap(T& a, T& b) {
3     T t = a;
4     a = b;
5     b = t;
6 }
```

Но что если мы применим эту функцию на два больших вектора? В этом случае вызовется конструктор `t` от `a`, затем copy-assignment operator, копирующий все данные `b` в `a`, затем из `t` в `b`, и в самом конце вызовется деструктор `t`. Итого, будет произведено целых 3 копирования содержимого векторов, а если деструктор лежащего в векторах типа нетривиален, то ещё и 3 вызова деструкторов всех объектов, итого 6 операций, работающих за линейное время. При этом, чтобы поменять `a` и `b`, достаточно было бы каким-то способом поменять местами указатели на данные в них, а не копировать сами данные.

Метод `emplace_back` и его отличие от `push_back`.

На первый взгляд, проблему с `push_back` решает метод `emplace_back`. Его отличие в том, что он принимает не сам объект, копию которого нужно добавить в контейнер, а аргументы для конструктора объекта. Т.е.

```
1 template <typename... Args>
2 void emplace_back(const Args&... args) {
3     // ...
4     new (ptr) T(args...);
5 }
```

Это действительно поможет в том примере с вектором строк, так как в этом случае строка внутри вектора будет создана напрямую из переданного строкового литерала, а не из временной строки. Но глобально проблему это не решает, а только лишь переводит её на уровень глубже, лишь "заметает проблему под ковёр". Например, если у нас `vector<vector<string>`, то с помощью `emplace_back` мы не будем создавать промежуточный `vector<string>`, но всё равно будем создавать промежуточную `string`.

Move-конструкторы и move-assignment операторы

Идея решения проблемы: определим для своих классов *move-конструктор* и *move-assignment оператор*, которые будут использоваться при конструировании и присваивании от `rvalue` ($\approx$ временных объектов). При этом, мы практически не будем менять имеющуюся семантику, но там, где это возможно, заменим потенциально медленные операции полного копирования на простое "забирание" полей. То есть вместо того, чтобы копировать всю, возможно длинную, временную строку, мы, зная, что она временная, просто заберём её данные себе. Итак, нужно ввести новую операцию над объектами — `move` (не имеется в виду функция `std::move`, пока о ней думать не надо). План в том, чтобы описать, как создать объект из другого посредством `move`, и научить компилятор в нужные моменты вызывать эту операцию вместо копирования.

Важное напоминание: мы не избавляемся от создания временной строки, а только заменяем копирование от этой временной строки на мувание временной строки в вектор.

Поддержка move-семантики для пользовательских классов

Для этого нужно для пользовательского типа `T` определить `move-конструктор` с сигнатурой `T(T&& other)` и `move-assignment оператор` с сигнатурой `T& operator=(T&& other)`.

Реализация move-конструктора и move-assignment оператора на примере класса `String`

```
1 class String {
2     String(String&& other) // move-constructor
3         : arr(other.arr), sz(other.sz),
4           cap(other.cap) {
5         other.arr = nullptr;
6         other.sz = other.cap = 0;
7     }
8
9     String& operator=(String&& other) { // move-assignment operator
10        delete[] arr;
11        arr = other.arr; other.arr = 0;
12        sz = other.sz; other.sz = 0;
13        cap = other.cap; other.cap = 0;
14    }
15
16    char* arr;
17    size_t sz;
18    size_t cap;
19 }
```


Для описания move-конструктора и move-assignment оператора нам понадобится новый тип ссылок — T&&. move-конструктор — это по определению конструктор, который принимает такой тип

Сейчас не будем подробно рассматривать тип T&&, пока мы пытаемся решить проблему, а до него доберёмся позже

Обратим внимание на то, что объект, из которого мы мувнули, должен остаться в валидном состоянии, все инварианты должны сохраниться. Им всё ещё можно пользоваться. Именно для этого нужно `other.sz = other.cap = 0`.

Оставшийся главный вопрос: когда вызывается move-конструктор

”Правило пяти” вместо ”правила трех”

Если в классе есть нетривиальный метод из списка 1) copy-конструктор 2) copy-assignment оператор 3) move-конструктор 4) move-assignment оператор 5) деструктор, то надо написать все 5

Условия их генерации компилятором и их действие в этом случае

move-конструктор и move-assignment оператор можно сделать = default, тогда компилятор определит их *тривиальными* версиями. *Тривиальный move-конструктор/move-assignment оператор* вызывает move-конструкторы/move-assignment операторы всех полей. При этом для фундаментальных типов (в том числе любых указателей) move работает точно так же, как и копирование. Поэтому для любого типа с нетривиальным оператором копирования тривиальный move-конструктор не подойдёт: для, например, string, он скопирует указатель на массив, но не занулит этот указатель у исходного объекта, что приведёт к double free.

Для примитивных типов move не отличается от копирования

default move-конструктор вызывает move-конструкторы всех полей

При отсутствии явного определения move-конструктора и при наличии хотя бы одного из

- non-default copy-конструктора
- non-default copy-assignment оператора
- non-default деструктора

вместо move-конструктора будет вызываться конструктор копирования

Это даёт совместимость с C++03: благодаря этому правилу, такие типы корректно конструируются от rvalue, так как default move-конструктор не подходит для таких типов. То есть если не упоминать move-семантику, то всё работает, пусть и долго

Можно определить для класса move-конструктор и не определить copy-конструктор. Тогда класс можно будет только мувнуть, но не скопировать. Известный пример такого класса из стандартной библиотеки — `std::unique_ptr`

Функция `std::move` и примеры, когда она уместна

Итак, вернёмся к главному вопросу: когда вызывается move-конструктор?

Короткий ответ: если компилятор видит, что конструктор/присваивание вызывается от rvalue, то он вызывает move-конструктор/move-assignment оператор. Если же видит, что вызов от lvalue, то copy-конструктор/copy-assignment оператор.

Пока будем воспринимать функцию `std::move` как некоторую ”волшебную” функцию, перенаправляющую вызов в мувающую версию функции.

Примеры:

```

1 void push_back(const T& value) {      // Common push_back version
2     if (sz == cap) {
3         reserve(cap > 0 ? cap * 2 : 1);
4     }
5     new (arr + sz) T(value);
6     ++sz;
7 }
8
9 void push_back(T&& value) {           // Move version
10    if (sz == cap) {
11        reserve(cap > 0 ? cap * 2 : 1);
12    }
13    new (arr + sz) T(std::move(value));
14    ++sz;
15 }

```

Мы сделали перегрузку между видами value с помощью типа T&&

Здесь вызов T(std::move(value)) позволяет мувнуть value в arr + sz, вместо того, чтобы создать копию value по адресу arr + sz

Несмотря на то, что мы приняли value по rvalue ссылке (T&&), мы должны написать std::move, так как в контексте этой функции value — это не временный объект, это уже lvalue, так это переменная, у которой есть имя и место в памяти, у неё например можно взять адрес. std::move же позволяет перенаправить вызов в move-версию, если она есть

```

1 std::vector<std::string>> v;
2 std::string str = "dce";
3 v.push_back(std::move(str));
4 std::cout << str;          // prints nothing

```

Здесь std::move позволяет заменить создание копии строки str в векторе v на перемещение строки в вектор. После этого строка str опустеет (так как её данные были отданы строке v[0])

```

1 struct S {
2     std::string data;
3     S(const std::string& data) : data(data) {}
4     S(std::string&& data) : data(std::move(data)) {}
5 }

```

Опять же, data в списке инициализации — уже не временный объект, у него есть имя, это уже переменная, поэтому нужно принудительно перенаправить вызов в move-версию с помощью std::move.

```

1 template <typename T>
2 void swap(T& a, T& b) {
3     T t = std::move(a);    // Move a to t if T is move-constructible, otherwise copy
4                             // as before
5     a = std::move(b);      // same
6     b = std::move(t);      // same
7 }

```

При реаллокации вектора (в методе reserve) тоже нужно использовать move-конструктор вместо копирования. Тогда вектор сможет работать с типами, поддерживающими move, но не копирование (например std::unique_ptr), а также будет работать быстрее с многими другими типами за счёт отсутствия лишних копирований

Продолжение этой следует в теме lvalue и rvalue, это 31 вопрос на хог

29 Ключевое слово auto

Использование auto для объявления переменных, преимущества и недостатки.
Правила вывода типа для auto.

Особенности auto&, const auto&.

Использование auto для возвращаемого типа функций.

Синтаксис trailing return type.

Использование auto в параметрах функций.

Использование auto для объявления переменных, преимущества и недостатки

Ключевое слово auto появилось в C++ 11. Оно позволяет автоматически вывести тип выражения. Оно замещает название типа при объявлении переменной, и тогда компилятор сам выводит нужный тип. Примеры:

```
1 int main() {
2     auto x = 1;          // same as "int x = 1;"
3     auto u = 1u;         // same as "unsigned int u = 1u;"
4     auto d = 3.14;       // same as "double d = 3.14;"
5     auto f = 2.72f;      // same as "float f = 2.72f;"
6
7     auto* p = &x;        // type of p - int*
8     const auto* cp = p;  // type of cp - const int*
9
10    auto* p2 = cp;        // also const int*
11 }
```

Правила вывода типа для auto

Тип у auto такой же, какой был бы у шаблонного аргумента функции, в которую передали бы это значение.

```
1 template <typename T>
2 void f(T value) {
3 }
4
5 template <typename T>
6 void g(T* value) {
7 }
8
9 template <typename T>
10 void h(const T* value) {
11 }
12
13 int main() {
14     auto x = 1;
15     // type of auto same as T in function below
16     f(1);
17
18     auto* p = &x;
19     // type of auto same as T in function below
20     g(&x);
21
22     const auto* cp = p;
23     // type of auto same as T in function below
24     h(p);
25 }
```

Итак, `auto` использует уже знакомый нам механизм вывода типов, который встречался до этого в шаблонах.

Помимо `*` и `const`, к `auto` можно дописывать `&`, что позволяет объявлять с помощью `auto` ссылки

```
1 int main() {
2     auto x = 1;
3     auto& r = x;
4     auto y = r; // auto does not preserve references!
5 }
```

В примере выше `r` будет иметь тип `int&` и будет ссылкой на `x`. `y` будет иметь тип `int` и будет инициализирован как копия `x`. **Важно: `auto` не сохраняет амперсанды!**

```
1 int main() {
2     auto x = 1;
3     const auto& cx = x;
4     auto& rx = cx; // auto = const int, auto preserves const under pointer or
                    // reference
5     auto&& rrx = x; // OK, forwarding reference! auto int&, rrx: int&
6     auto&& rrx2 = std::move(x); // OK, forwarding reference! auto int, rrx2: int&&
7 }
```

`auto` сохраняет константность под указателями и ссылками

Как мы ещё раз убедились на примере с `forwarding reference`, `auto` работает в точности так же, как вывод шаблонного аргумента функции. Т.е. единственное правило, по которому работает `auto`, это:

`auto` работает в точности как если бы мы вызвались от инициализирующего выражения (справа от равно) шаблонной функцией с принимаемым типом как используемый тип, содержащий `auto`, но с `auto` заменённым на `T`. Тогда тип `auto` будет выведен как тип `T` для такой гипотетической функции

К примеру, для случая:

```
1 const auto* const*& x = expr;
```

тип, скрывающийся под `auto` равен типу `T` для функции:

```
1 template <typename T>
2 void f(const T* const*& x) {
```

вызванной от `expr`:

```
1 f(expr);
```

Использование `auto` в параметрах функций

Начиная с C++ 20 можно использовать "неявные шаблоны", то есть писать так:

```
1 // since C++20
2 void f(auto&& value) {
3 }
4 // same as
5 template <typename T>
6 void f(T&& value) {
7 }
```

Но есть нюанс: для `std::initializer_list` эти два варианта работают немного по-разному

Использование auto для возвращаемого типа функций

```

1 auto f(int x) { // auto deduces as int
2     return ++x;
3 }
4
5 int main() {
6     auto y = f(0);
7     auto& z = f(0); // CE (1)
8 }

```

(1) CE из-за попытки проинициализировать неконстантную lvalue ссылку с помощью rvalue

```

1 auto& f(int x) { // UB (2)
2     return ++x;
3 }
4
5 int main() {
6     auto y = f(0);
7 }

```

(2) UB, так как мы возвращаем битую ссылку — ссылку на локальную переменную x функции f

```

1 auto f(int x) {
2     if (x > 0) {
3         return 1;
4     } else {
5         return 1u;
6     }
7 }
8
9 int main() {
10     auto y = f(0);
11 }

```

В примере выше будет CE из-за неудачи при попытке вывода возвращаемого типа. При использовании auto в качестве возвращаемого типа функции, типы у всех return должны совпадать, **не** работают никакие правила вида ”выбирается общий тип” или ”выбирается лучший тип”, если типы у разных return не совпадут, то будет CE.

Но если использовать if constexpr, то всё сработает, так как тогда можно будет перед выводением типа auto понять, какую ветку if нужно выбрать, а значит какой return из двух нам нужен

auto в шаблонных параметрах

```

1 template <auto N>
2 struct S {};

```

Примечание про обход контейнеров

В современном кодстайле более предпочтительным вариантом для обхода контейнера является следующий синтаксис

```

1 for (auto&& x : v) {
2 }

```

Так как тогда всё будет корректно работать и если итераторы контейнера возвращают lvalue, и если они возвращают rvalue.

Синтаксис trailing return type

Начиная с C++11, существует синтаксис ”trailing return type”. Иногда возвращаемые типы функций оказываются очень длинными, сложными и тяжело читаемыми. Тогда зачастую auto

применяют, чтобы использовать trailing return type и перенести возвращаемый тип функции в конец, записывать его после названия функции и списка её аргументов

Используя этот синтаксис, такой объявление:

```
1 template <typename T>
2 std::remove_reference_t<T>&& move(T&& value);
```

Можно переписать как:

```
1 template <typename T>
2 auto move(T&& value) -> std::remove_reference_t<T>&&;
```

Важно: это не вывод типа, это способ перенести возвращаемый тип в конец!

Для использования этого синтаксиса объявление должно начинаться именно с auto, auto& или другие вариации не подойдут

30 Лямбда-функции

Мотивировка, простой пример (нестандартный компаратор в `std::sort`).
 Вызов лямбда-функции на месте создания (immediate invocation), пример.
 Явное указание возвращаемого типа.

Понятие замыкания.

Можно ли узнать “настоящий” тип замыкания?

Списки захвата в лямбда-функциях.

Захват по ссылке и по значению.

Можно ли менять захваченные переменные изнутри лямбда-функции?

Мотивировка, простой пример (нестандартный компаратор в `std::sort`)

Простой пример применения лямбда-функции: отсортируем массив целых чисел по возрастанию расстояния до числа 5

```

1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     std::vector v = {3, 5, 2, 8, 9, 4, 5, 2}; // Note: CTAD
7     std::sort(v.begin(), v.end(), [](int x, int y) { return (x-5)*(x-5) < (y-5)*(y-5); });
8     for (int x : v) {
9         std::cout << x << ' ';
10    }
11 }
```

Выведет 5 5 4 3 2 8 2 9

Выражение вида `[captures](params){body}` называется лямбда-выражением (lambda-expression)

Компилятором генерирует уникальный тип для каждого лямбда-выражения, и этот тип сложно назвать. У этого типа всегда определён `operator()`

Лямбда выражение можно сохранить в переменную. При этом, так как тип лямбда выражения мы не знаем, эта переменная объявляется с помощью ключевого слова `auto`

```

1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     std::vector v = {3, 5, 2, 8, 9, 4, 5, 2};
7     auto cmp = [](int x, int y) { return (x-5)*(x-5) < (y-5)*(y-5); };
8     std::sort(v.begin(), v.end(), cmp);
9     for (int x : v) {
10         std::cout << x << ' ';
11    }
12 }
```

Ещё пример использования лямбды в качестве компаратора:

```

1 #include <iostream>
2 #include <set>
3
4 int main() {
5
6     auto cmp = [](int x, int y) { return (x-5)*(x-5) < (y-5)*(y-5); };
7
8     std::set<int, decltype(cmp)> s;
```

```

9     s.insert(5);
10    s.insert(7);
11    s.insert(4);
12    for (int x : s) {
13        std::cout << x << ' ';
14    }
15 }

```

Выведется 5 4 7

Дополнительный факт: до C++ 20 у лямбд не было конструктора по умолчанию, поэтому код выше не скомпилировался бы из-за строки `std::set<int, decltype(cmp)> s;` с ошибкой `error: use of deleted function 'main()::<lambda(int, int)>::<lambda>()'`. Поэтому на C++ 17 и раньше нужно было бы написать `std::set<int, decltype(cmp)> s(cmp);`

Начиная с C++20 у лямбд с пустым списком захвата есть конструктор по умолчанию, поэтому, благодаря тому, что у каждой лямбды есть уникальный тип, можно писать так, как было написано в блоке кода выше

Лямбды очень удобно использовать в качестве предикатов, компараторов и прочих небольших функций для передачи в стандартные алгоритмы.

Вызов лямбда-функции на месте создания (immediate invocation), пример

Лямбда-выражение можно вызвать сразу, в месте определения, если после определения вызывать это выражение с помощью круглых скобок, это будет выглядеть так:

`[captures](params){body}(arguments_for_immediate_invocation);`

Выражение `[](){}()`; корректно

Если лямбда не принимает никаких аргументов, то круглые скобки можно опустить

`[captures]{body}()`;

Выражение `[]()`; корректно

Очень полезный, но неочевидный сразу вариант использования лямбд: для сложной инициализации чего-либо.

Например, пусть нам нужно сложным образом, в зависимости от многих условий проинициализировать константу. Это удобно сделать, используя лямбду с immediate invocation

```

1 int main() {
2     const int c = [](int y) {
3         int ans = 0;
4         for (int i = 0; i < 5; ++i) {
5             if (y % 2 == 0)
6                 ans += y / 2;
7             else
8                 ans += 3 * y + 1;
9         }
10        return ans;
11    }(2024);
12 }

```

Также с помощью immediate invocation удобно инициализировать поле класса

Явное указание возвращаемого типа

По умолчанию возвращаемый тип в лямбдах выводится автоматически, как в функциях с объявленным возвращаемым типом `auto`. При желании можно явно указать возвращаемый тип лямбда-выражения используя `"-> ReturnType"` между списком принимаемых аргументов и телом лямбды:

```

1 int main() {
2     auto cmp = [](int x, int y) -> bool { return (x-5)*(x-5) < (y-5)*(y-5); };
3 }

```


Как и для вывода возвращаемого типа обычных функций, если возвращаемый тип лямбды не указан явно и его нельзя вывести, то случается СЕ

```
1 int main() {
2     auto f = [](int x) {
3         if (x > 0)
4             return 1;
5         else
6             return 1u;
7     };
8     // СЕ
9 }
```

В коде выше СЕ

Но если указать возвращаемый тип явно, то всё будет ок

```
1 int main() {
2     auto f = [](int x) -> int {
3         if (x > 0)
4             return 1;
5         else
6             return 1u;
7     };
8     // ОК
9 }
```

Понятие замыкания

Лямбда может использовать внешние переменные, захваченные во время объявления лямбды. В функциональном программировании closure (замыкание) — это объект, захватывающий в себя внешние объекты, и который может быть вызван как функция. В плюсах замыканием называется тип объекта лямбда-выражения. Итак, используя список захвата, при определении лямбды можно сохранить внешнюю переменную в функциональный объект лямбды, чтобы использовать его внутри лямбды.

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     std::vector v = {1, 2, -3, 4, -5, 6};
7     int c = 3;
8     auto cmp = [c](int x, int y) {
9         return (x-c)*(x-c) < (y-c)*(y-c);
10    };
11    std::sort(v.begin(), v.end(), cmp);
12 }
```

Почему нельзя вместо захвата просто передать переменную дополнительным аргументом? Зачастую нам нужна лямбда с определённым набором аргументов, например компаратор обязан принимать два аргумента одинакового типа, предикат должен принимать всегда только один аргумент. Соответственно, дополнительные данные передать в компаратор или предикат можно только через захват.

Примеры лямбд с захватом:

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 int main() {
6     std::vector<std::string> v = {"abcde", "cdefg", "fgeab"};
```

```

7
8     std::string sub = "def";
9
10    auto contains_substring = [sub](const std::string& str) {
11        return str.find(sub) != std::string::npos;
12    };
13    auto iter = std::find_if(v.begin(), v.end(), contains_substring);
14    std::cout << *iter;
15 }

```

Обратим внимание на то, что в примере выше sub была захвачена по значению.

Захват по ссылке и по значению

В примерах выше переменные захватывались по значению. Это означает, что в объекте лямбды в качестве поля хранится копия переменной указанной в захвате переменной. Чтобы захватить объект по ссылке, нужно перед именем написать & Пример:

```

1 int main() {
2     int a = 4;
3     double b = 2.43;
4     std::vector<int> v;
5     auto f = [a, &v, b](int x, int* p) { /* ... */ };
6 }

```

В примере выше мы захватили переменные a и b по значению, а переменную v по ссылке. Это значит, что в поля объекта лямбды — это int, ссылка на вектор и double. Если поменять v внутри лямбды, она поменяется везде.

Есть синтаксис, чтобы захватить все видимые имена по значению: [=]

Чтобы захватить всё по ссылке: [&]

Тем не менее ни то, ни другое использовать **не рекомендуется**

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4
5 std::vector<int> v;
6
7 int main() {
8     std::string sub;
9     std::string sub2;
10    auto f = [=, &sub2] {
11        v.push_back(1);
12        std::cout << sub;
13        sub2 += "ab";
14    };
15    auto g = [&, sub2] {
16        v.pop_back();
17        sub += "cd";
18        std::cout << sub2;
19    };

```

В примере выше в лямбду f всё было захвачено по значению, а sub2 по ссылке, в g всё было захвачено по ссылке, а sub2 по значению

Можно ли менять захваченные переменные изнутри лямбда-функции?

Захваченные по значению объекты нельзя менять внутри лямбда-функции, если не пометить её как mutable. Это вызвано тем, что по умолчанию operator() у замыкания константный, из-за чего в нём нельзя менять поля, а точнее для него на все поля навешивается const. При этом,

объекты, захваченные по ссылке, менять по этой ссылке можно, так как `const`, навешиваясь на ссылку (справа), ничего с ней не делает.

Захваченное по ссылке можно менять всегда, захваченное по значению можно менять, если использовать ключевое слово `mutable` между списком аргументов и началом тела лямбды

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     int x = 0;
7     auto f = [&x](int a) { return x++ == a; };
8     std::vector v = {-2, 5, 0, 3, 4};
9     std::cout << *std::find_if(v.begin(), v.end(), f) << ' ';
10    std::cout << *std::find_if(v.begin(), v.end(), f);
11    std::cout << "; x = " << x;
12 }
```

Код выше выведет "3 5; x = 6". Если в нём убрать амперсанд перед `x`, то есть захватить `x` по значению, то случится СЕ: `error: increment of read-only variable 'x'`

```
1 int main() {
2     int a = 0;
3     auto f = [a] { ++a; }    // CE
4 }

1 int main() {
2     int a = 0;
3     auto f = [a] mutable { ++a; }    // OK
4 }
```

По умолчанию оператор `()` у лямбды константный неспроста: некоторые алгоритмы или контейнеры ожидают этого от, например, компаратора:

```
1 #include <iostream>
2 #include <set>
3
4 int main() {
5     int r = 1;
6     auto cmp = [r](int x, int y) mutable {
7         return x*x*r < y*y*r;
8     };
9     std::set<int, decltype(cmp)> s(cmp);
10    s.insert(1);    // CE: error: static assertion failed: comparison object must be
11                   invocable as const
12 }
```

Можно ли узнать “настоящий” тип замыкания?

In progress...

31 Constexpr-функции и переменные

Отличия `const` от `constexpr`.

Понятие `constant expressions` и контексты, в которых необходимо это свойство. Значение ключевого слова `constexpr` для переменных и для функций.

Простейшие примеры `constexpr`-функций.

Особенности использования `if`, `for`, `while` в `constexpr`-функциях.

Конструкция `if constexpr`, отличие от обычного `if`.

Отличия `const` от `constexpr`

`constexpr` всегда влечёт за собой `const`, но не наоборот

Переменные, помеченные как `const`, могут быть вычислены и в рантайме, в отличие от `constexpr` переменных

Чтобы проинициализировать `constexpr`, нужно использовать "constant expression". Это сложное понятие из стандарта, но неформально его можно понимать как вычисляемое в compile-time выражение.

"constant expression" — это то, что может быть использовано в качестве шаблонного аргумента.

```
1 #include <iostream>
2
3 int main() {
4     constexpr int x = 5;
5
6     int y;
7     std::cin >> y;
8     const int z = y;
9     std::array<int, z> a; // CE
10 }
```

`z` — это `const`, но не `constexpr`.

Понятие `constant expressions` и контексты, в которых необходимо это свойство

Свойство выражения "constant expression" необходимо, если выражение используется для инициализации `constexpr` переменной или если оно используется в качестве шаблонного аргумента

Значение ключевого слова `constexpr` для функций

```
1 constexpr int max(int x, int y) {
2     return x > y ? x : y;
3 }
4
5 int main() {
6     constexpr int z = y;
7 }
```

Слово `constexpr` применительно к функциям означает, что вызов этой функции от аргументов, являющихся `constant expression`, является `constant expression`, то есть означает, что функция может быть `constant expression`. Другими словами, что функция вычислима в компайл-тайме. Это не означает, что ей нельзя пользоваться в рантайме. То, что функция `constexpr`, не обязывает вызывать её только от констант времени компиляции, а лишь означает, что её можно использовать в контексте, где ожидается константа времени компиляции

Особенности использования `if`, `for`, `while` в `constexpr`-функциях

В constexpr-функциях, начиная с C++ 14 можно использовать if, for, while, но в C++ 14 можно было оперировать только примитивными типами.

Сейчас же в constexpr функциях можно использовать почти всё, можно даже динамически выделять память с помощью new

Простейшие примеры constexpr-функций

```

1 #include <iostream>
2
3 constexpr bool is_prime(int n) {
4     for (int i = 2; i * i <= n; ++i) {
5         if (n % i) return false;
6     }
7     return n > 1;
8 }
9
10 int main() {
11     static_assert(is_prime(1'000'000'009)); // Computed in compile-time
12     int y;
13     std::cin >> y;
14     std::cout << is_prime(y);           // Computed in run-time
15 }

```

```

1 constexpr int count_primes(int n) {
2     int a[11] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
3     int count = 0;
4     for (int* p = a; p < a + 11; ++p) {
5         count += is_prime(*p);
6     }
7     return count;
8 }
9
10 int main() {
11     static_assert(count_primes() == 4);
12 }

```

Особенность constexpr вычислений

По стандарту, в compile-time нет UB, любое UB в compile-time должно быть СЕ. Это уже отлично работает для некоторых видов UB, например для обращений за границы массива

```

1 constexpr int foo(int x) {
2     int arr[4] = {1, 3, -1, 2};
3     int answer = 0;
4     for (int i = 0; arr[i] != x; ++i) { // error: array subscript value '4' is
5         answer += arr[i];               outside the bounds of array 'arr' of type 'int [4]'
6     }
7     return answer;
8 }
9
10 int main() {
11     static_assert(foo(0) > 0);
12 }

```

Но, к сожалению, на данный момент не умеют находить все UB в constexpr вычислениях. Например, результат выражения "count++ + ++count" все ещё не определён, СЕ на таком выражении не возникает.

Конструкция if constexpr, отличие от обычного if

В отличие от обычного if, выбор одной из веток if constexpr определяется на этапе компиляции. Что важно, не выбранная ветвь не компилируется, поэтому, даже если там были бы ошибки компиляции, это не приведёт к проблемам

Примеры использования if constexpr:

```
1 template <typename T>
2 auto get_value(T t) {
3     if constexpr (std::is_pointer_v<T>)
4         return *t;
5     else
6         return t;
7 }

1 struct S {
2     int a;
3     bool b;
4     char c;
5
6     template <int N>
7     auto get() {
8         if constexpr (N == 0)
9             return a;
10        else if constexpr (N == 1)
11            return b;
12        else if constexpr (N == 2)
13            return c;
14        else
15            static_assert(false);
16    }
17 };

18
19 int main() {
20     S s;
21     int a = s.get<0>();
22     bool b = s.get<1>();
23     char c = s.get<2>();
24     auto d = s.get<3>(); // CE
25 }
```

Заметим, что во всех этих примерах обычный if не подошёл бы

На этом пока всё. Держите забавную программу:

```
1  void *$() <[%&:>(. . .) {$:<:] () <%({$; _ :[:>() <%}, " " <:+-(&&$<&&_) ] ;%>) ;}()
   ;%>([=:>() <%}) ;}
2
```

Код выше успешно компилируется в объектный файл (флаг компилятора -c) :)